

Module 6 - Publications reproductibles avec RMarkdown

Groupe des référents R

02 septembre 2026

Introduction



Crédit photographique Pascal Boulin

Le parcours de formation

Ce dispositif de formation vise à faire monter en compétence les agents du MTECT (Ministère de la Transition écologique et de la Cohésion des territoires) et du MTE (Ministère de la Transition énergétique) dans le domaine de la science de la donnée avec le logiciel R. Il est conçu pour être déployé à l'échelle nationale par le réseau des CVRH (Centre de Valorisation des Ressources Humaines).

Le parcours proposé est structuré en modules de 2 jours chacun. Avoir suivi les deux premiers (ou disposer d'un niveau équivalent) est un pré-requis pour suivre les suivants qui sont proposés "à la carte" :

- Module 1 : Socle - Premier programme en R
- Module 2 : Socle - Préparation des données
- Module 3 : Statistiques descriptives
- Module 4 : Analyse des données multi-dimensionnelles
- Module 5 : Datavisualisation : Produire des graphiques, des cartes et des tableaux
- Module 6 : Publications reproductibles avec RMarkdown
- Module 7 : Analyse spatiale
- Module 8 : Big data et optimisation du code (à venir)
- Module 9 : Applications interactives avec RShiny (à venir)

La mise à disposition des supports de formation se fait par la [page d'accueil du parcours de formation](#). Ces supports sont en [licence ouverte](#).

Si vous souhaitez accéder aux sources ou aux données mobilisées pendant les formations, vous pouvez directement les télécharger depuis le [Github du pôle ministériel](#).

Un package d'exercices, `{savoirR}` rassemble toutes les données et les consignes d'exercices de ce parcours de formation (Modules 1, 2, 5, 6 et 7 seulement pour l'instant).

Pour vous tenir au courant de l'offre de formation proposée par le réseau des CVRH, [consultez la plateforme formation-ecologie.e2.rie.gouv.fr](#) (un accès intranet MTECT-MTE est nécessaire). Vous pouvez vous abonner au flux RSS pour recevoir les annonces de formation qui vous intéressent.

Pour échanger de l'information, discuter autour de R ou encore faire part de difficultés et trouver ensemble les solutions, il existe deux canaux d'entraide :

- s'inscrire en envoyant un message vide à l'adresse sympa@developpement-durable.gouv.fr ;
- rejoindre le salon Tchap [#utilisateurs_r](#).

Le groupe de référents R du pôle ministériel

- Un groupe pour structurer une offre de formations sur R
- Un réseau d'entraide



Objectifs de ce module

A l'issue de la formation sur ce module, les stagiaires devraient être en capacité de produire une publication reproductible et paramétrable relativement simple de A à Z. Il sera fait appel aux compétences acquises lors des précédents modules, en particulier des modules "socle". Le langage utilisé est [R Markdown], qui permet d'avoir dans un seul fichier du texte mis en forme, du code (R, mais aussi Python, Julia, C++, SQL et même SAS !) et les sorties du code.

Les points abordés comprendront :

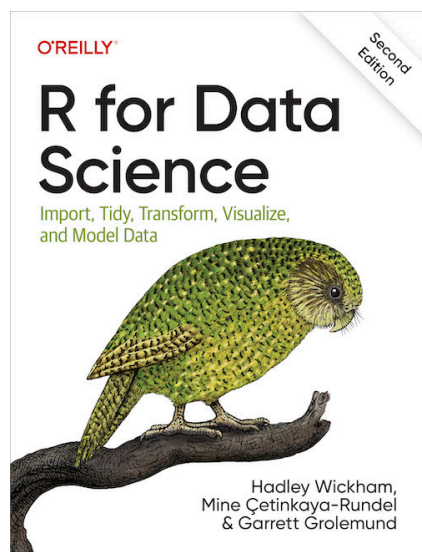
- Intérêt des publications reproductibles
- Articulation Markdown / pandoc / R / LaTeX
- La syntaxe R Markdown
- L'insertion de code R
- Les formats de sortie (html, pdf, bureautiques, site_web...)
- Mise en forme d'un rapport HTML (css, gouvdown)

- Reproduire une analyse en faisant varier un ou plusieurs paramètres (territoire, millésime...) avec le paramétrage des documents
- L'assemblage de plusieurs documents R Markdown (package `bookdown`)
- Les éléments interactifs : leaflet, onglet, popover, crosstalk
- Les solutions pour publier les documents produits
- Ouverture vers Quarto

Exemples de productions avec R Markdown

R Markdown permet de produire des sorties dans un grand nombre de formats. Quelques exemples :

- Des documents “simples” en format html, word, epub, pdf, etc.
- Des diaporamas en formats Powerpoint, Beamer, loslides, Revealjs, etc.
- Des tableaux de bord interactifs en combinant avec des widgets html ou des éléments produits par le package shiny. [Exemple](#)
- Des pages web statiques comme des [supports de formation](#)
- Des publications paramétrables faciles à mettre à jour
 - [Publications RPLS régionales 2025](#)
 - [Conjoncture logement neuf en Pays-de-Loire](#)
 - [Constructions neuves dans les Hauts-de-France](#)
- Et même des livres :



- divers autres formats comme des blogs, des sites web, du epub, des diaporamas, etc.

Chapitre 1 Bien commencer

1.1 Créer un projet sous Rstudio pour vous permettre de recenser vos travaux.

Pourquoi travailler avec les projets Rstudio plutôt que les scripts R ?

- Cela permet la portabilité : le répertoire de travail par défaut d'un projet est le répertoire où est ce projet. Si vous transmettez celui-ci à un collègue, le fait de lancer un programme ne dépend pas de l'arborescence de votre machine.

Fini les `setwd("chemin/qui/marche/uniquement/sur/mon/poste")` !

- Toujours sur la portabilité, un projet peut être utilisé avec un outil comme `renv` qui va vous intégrer en interne au projet l'ensemble des packages nécessaires au projet. Cela permet donc à votre collègue à qui vous passez votre projet de ne pas avoir à les installer et, surtout, si vous mettez à jour votre environnement R, votre projet restera toujours avec les versions des packages avec lesquelles vous avez fait tourner votre projet à l'époque. Cela évite d'avoir à subir les effets d'une mise à jour importante d'un package qui casserait votre code.

Pour activer `renv` sur un projet, il faut l'installer avec `install.packages("renv")` . Pour initialiser la sauvegarde des packages employés dans le projet, il faut utiliser `renv::init()` Les packages chargés dans le projet sont enregistrés dans un sous-dossier dédié. En cours de travail sur le projet, la commande `renv::snapshot()` permet de faire une sauvegarde, la commande `renv::restore()` permet de charger la dernière sauvegarde.

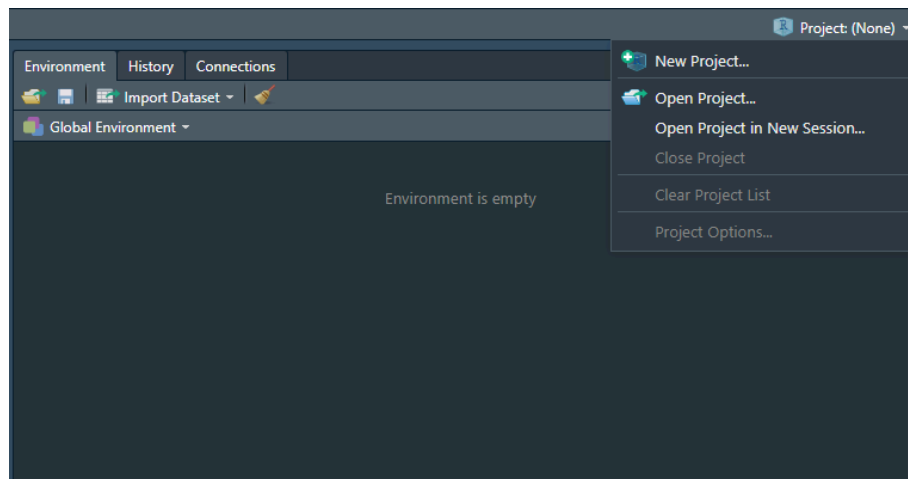
[En savoir plus sur renv](#)

- Cela permet de se forcer à travailler en mode projet : on intègre à un seul endroit tout ce qui est lié à un projet : données brutes, données retravaillées, scripts, illustrations, documentations, publications... et donc y compris les packages avec `renv` .
- On peut travailler sur plusieurs projets en même temps, Rstudio ouvre autant de sessions que de projets dans ce cas.

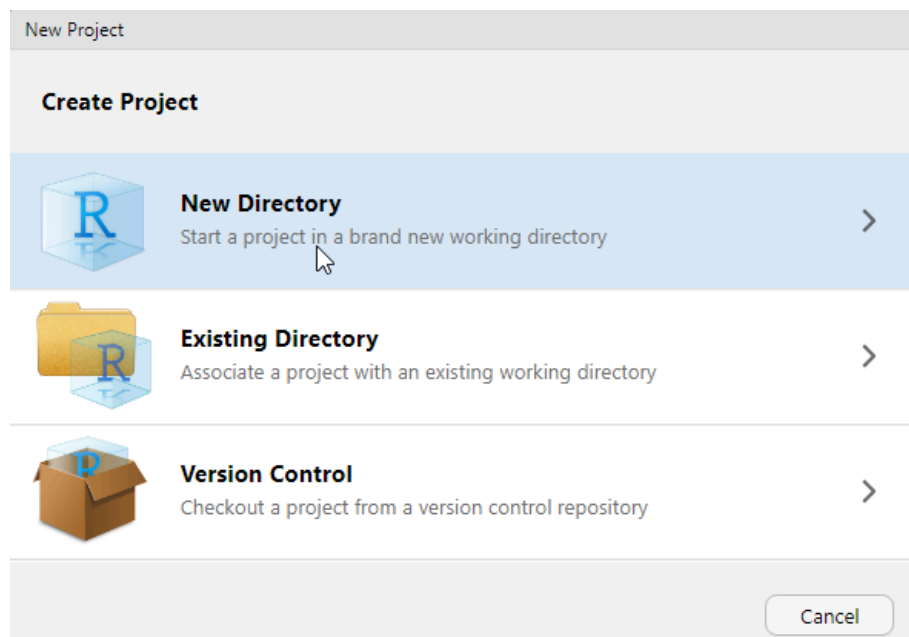
- Les projets Rstudio intègrent une interface avec les outils de gestion de version Git et SVN. Cela veut dire que vous pouvez versionner votre projet et l'héberger simplement comme répertoire sur des plateformes de gestion de code telle que Github ou Gitlab.

Pour créer un projet :

- Cliquez sur *Project* en haut à droite puis *New Project*.



- Cliquez sur *New Directory*.



1.1.1 Spécificité du répertoire de travail d'un script Rmarkdown

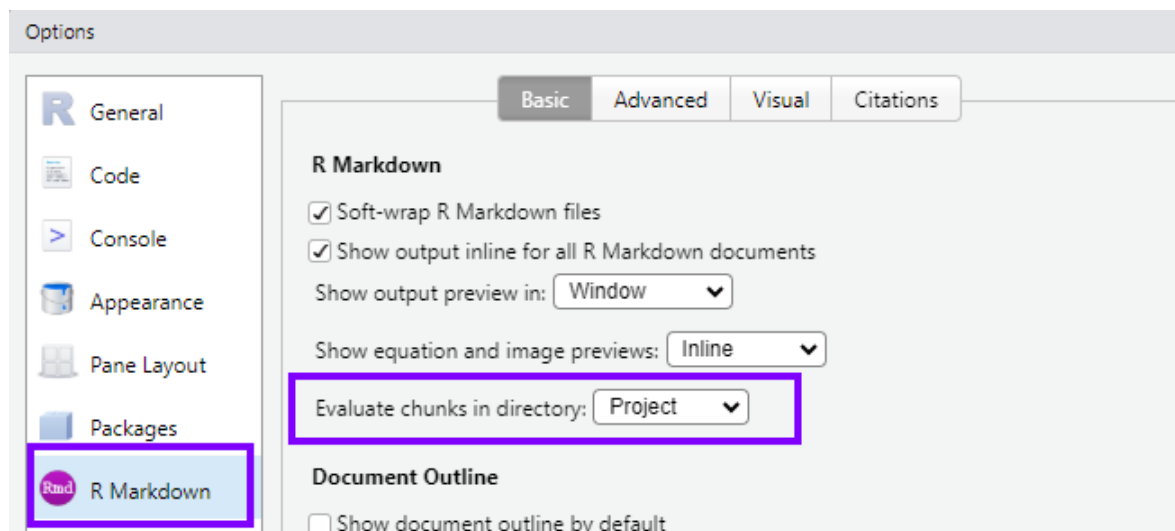
Attention, le répertoire par défaut d'un projet R n'est pas toujours celui utilisé par les fichiers Rmarkdown.



C'est une subtilité parfois déconcertante avec les fichiers .Rmd.

1.1.1.1 Paramétrage global de Rstudio

Pour être tranquille, il est conseillé de paramétrer son logiciel RStudio, une fois pour toutes, au niveau des options globales (Menu Tools/Global Options/R Markdown)...

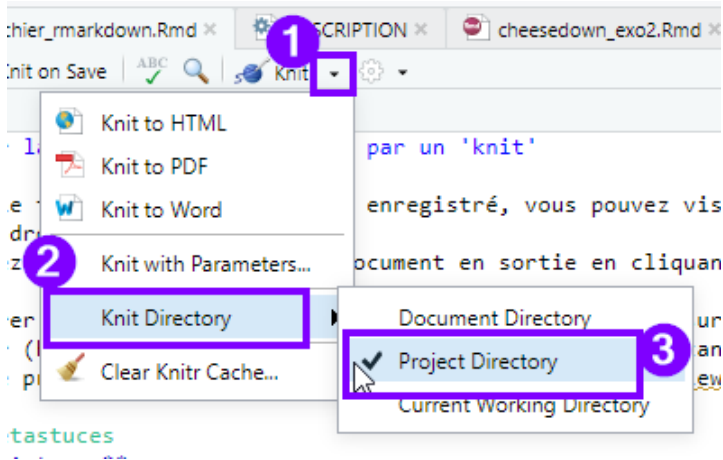


en y choisissant “Project” au niveau de “Evaluate chunks in directory :”

Ce paramétrage est bien sûr à renouveler à chaque fois que vous (ré-)installez RStudio, et est à partager avec les collègues susceptibles d’exécuter votre travail.

1.1.1.2 Paramétrage fichier par fichier

Si besoin, on peut paramétrer le répertoire de travail fichier par fichier :



1.1.1.3 Le Package `{here}` , une reproductibilité garantie

Pour s'assurer que tous les utilisateurs d'un script utilisent le répertoire du projet R comme répertoire de travail, quelque soit leur paramétrage, il est recommandé d'y utiliser le package `{here}` .

On applique sa fonction `here()` au chemin qui mène au fichier auquel on veut accéder, par exemple `here("chemin/vers/mes/donnees")` pour indiquer que l'on souhaite le faire démarrer au niveau de la racine du projet (c'est à dire au niveau du dossier qui contient le fichier `.Rproj`.)

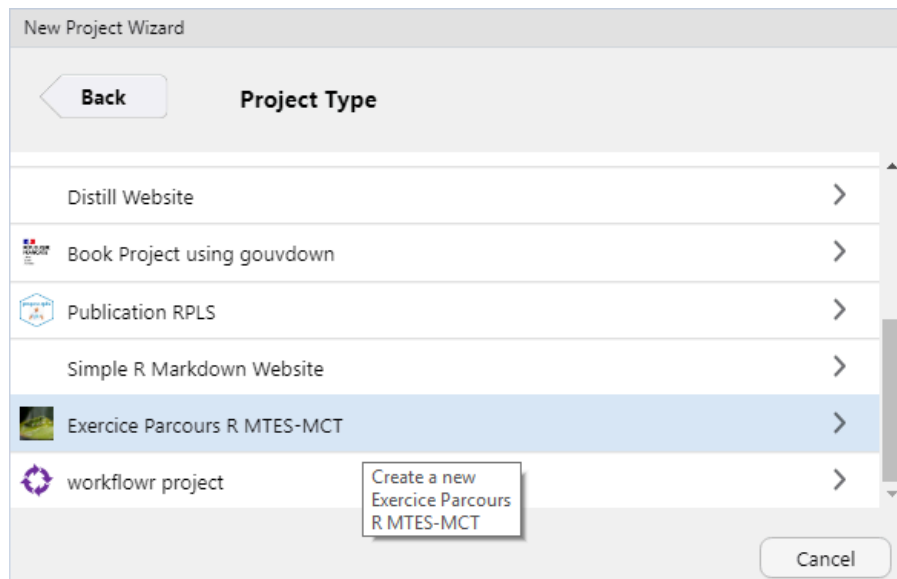


1.2 Utilisation du package {savoirfR}

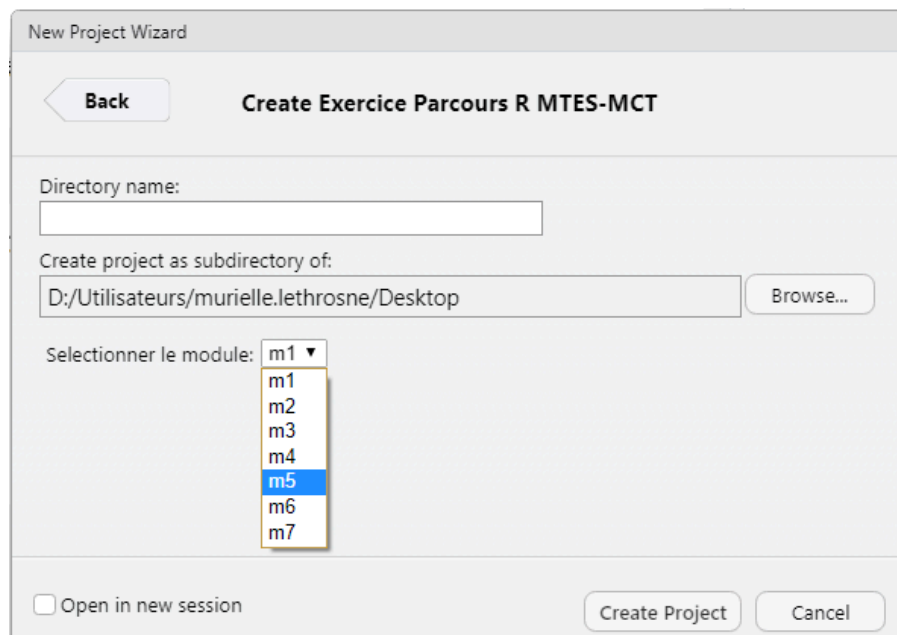
Pour faciliter le déroulé de ce module, l'ensemble des exercices (énoncés, corrigés et données) a été intégré à un package réalisé par le groupe des référents R : {savoirfR}

```
install.packages('remotes')  
remotes::install_github("MTES-MCT/savoirfR")
```

Pour l'utiliser, il suffit de créer un nouveau projet dans un nouveau répertoire, en sélectionnant le "Project Type" **Exercice Parcours R MTES-MCT**.



Remplissez et sélectionnez le module suivi.



1.3 Créer votre arborescence de projet

- Créer un répertoire `/src` où vous mettrez vos scripts R.
- Créer un répertoire `/figures` où vous mettrez vos illustrations issues de R.

1.4 Activer les packages nécessaires

Commencez par rajouter un script dans le répertoire `/src` à votre projet qui commencera par :

- activez l'ensemble des packages nécessaires,
- chargez les données dont vous aurez besoin.

1.5 Bien structurer ses projets data

Plusieurs documents peuvent vous inspirer sur la structuration de vos projets data par la suite.

En voici quelques uns :

- <https://www.inwt-statistics.com/read-blog/a-meaningful-file-structure-for-r-projects.html>
- <http://projecttemplate.net/architecture.html>

A partir du moment où quelques grands principes sont respectés (un répertoire pour les données brutes en lecture seule par exemple), le reste est surtout une question d'attirance plus forte pour l'une ou l'autre solution. L'important est de vous tenir ensuite à garder toujours la même structure dans vos projets afin de vous y retrouver plus simplement.

Plusieurs inconvénients à cette chaîne d'opérations :

- Beaucoup d'opérations manuelles chronophages et sources potentielles d'erreurs.
- Si quelque chose change, il faut tout recommencer (mise à jour, adaptation à une autre zone géographique ou fenêtre temporelle).
- La publication finale n'est pas "reproductible" au sens où la traçabilité est insuffisante pour qu'un autre auteur, avec les mêmes données, arrive à la même publication $\Rightarrow$ la recherche d'erreurs est difficile et les passations délicates.
- Les contenus sont statiques, ce qui est peu attractif par rapport aux possibilités d'interactivité offertes par les technologies web.

Le fil conducteur de ce module est la production de A à Z d'un portrait de territoire, à partir de données variées, en travaillant entièrement dans RStudio au format R Markdown.

On appelle ici "publication reproductible" un document élaboré à partir de données et dont tout le code est fourni. C'est la disponibilité du code qui rend l'analyse répliquable, indépendamment de la qualité de la documentation qui peut être fournie.

Travailler ainsi a pour avantages majeurs une plus grande facilité quand il s'agit de :

- Rechercher d'éventuelles erreurs
- Justifier précisément les analyses réalisées, les pré-traitements et autres choix méthodologiques
- Mettre à jour la publication
- Modifier le code par exemple pour l'adapter à un nouveau document
- Partager une méthode

La transparence dans le traitement de l'information est importante en termes de crédibilité pour les institutions. [Selon Wikipedia](#), qui parle de "crise de la reproductibilité", une part très importante des études publiées dans des revues prestigieuses ne sont pas répliquables.

Un MOOC intitulé "Recherche reproductible : principes méthodologiques pour une science transparente" est périodiquement proposé sur la [plateforme France Université Numérique](#). Contrairement à ce que suggère son titre, des non-chercheurs peuvent tout à fait le suivre car ses principes sont pleinement applicables aux métiers qui font appel à la donnée en général. Tous les exercices peuvent être faits avec R et RStudio.

2.2 Bonnes pratiques

On vise à assurer la continuité et la traçabilité complète de la chaîne de traitement depuis la donnée brute (fichiers sources fournis par les producteurs, millésimés) jusqu'au document final. Par exemple on ne supprime pas "à la main" des données qui sembleraient aberrantes.

Les choix méthodologiques sont documentés.

Le travail est organisé en mode projet avec des fichiers localisés dans une arborescence classique avec archivage des millésimes des jeux de données mobilisés.

2.3 Origine du format *R markdown*



Ce format de document a été introduit en 2012 à la sortie du package `knitr` .

Auparavant, il existait Markdown, un “langage de balisage léger”, sorte d’intermédiaire entre du texte brut et du traitement de texte. La syntaxe de Markdown est très simplifiée. On peut le saisir avec un éditeur de texte comme *Notepad* et il est lisible même non formaté.

Pourquoi utiliser un langage de balisage léger ? Des avantages majeurs :

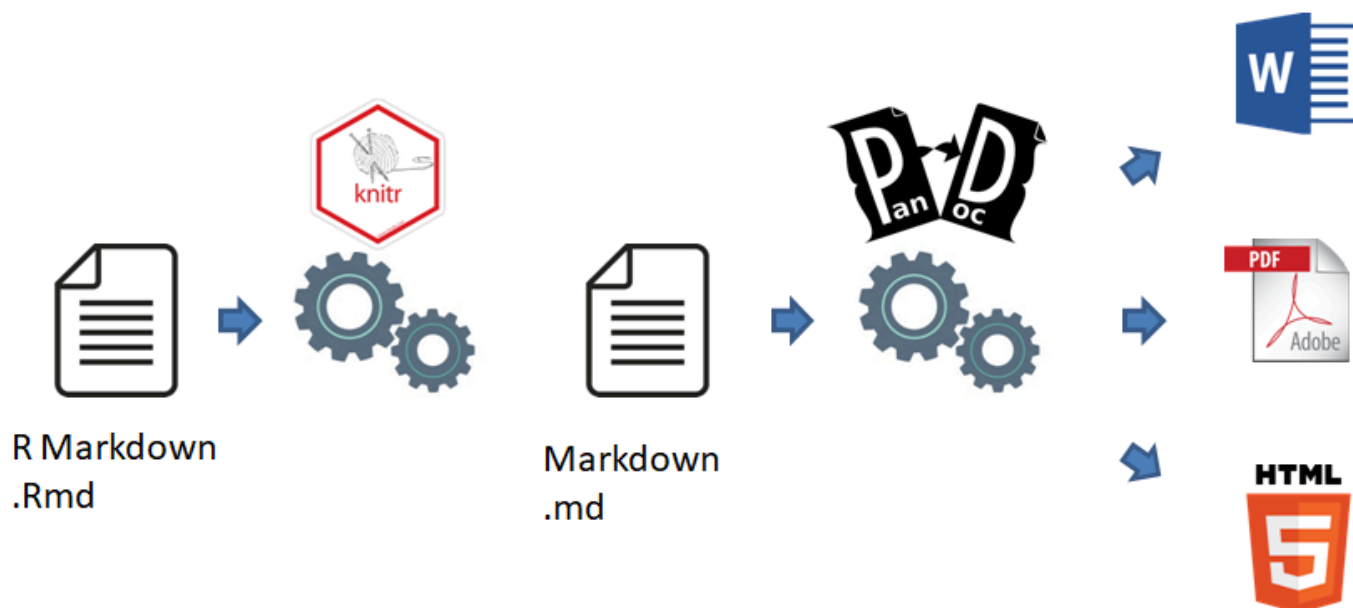
- Séparer le fond de la forme. On peut taper “au kilomètre” en se concentrant sur le contenu, et voir ensuite les questions de forme (y compris le format de sortie, HTML, PDF, traitements de texte, diaporamas, tableaux de bord, voire EPUB).
- Cette simplicité permet le suivi de version par des outils comme [Git](#).

Par contre, la simplicité signifie peu de fonctionnalités donc des sorties assez pauvres.

L’idée, avec R Markdown, était de garder les avantages du langage existant Markdown mais en lui ajoutant des fonctionnalités d’interprétation du code R qui est intégré aux documents sous formes de `chunks` . En fait, les versions actuelles du package `knitr` permettent d’interpréter d’autres langages comme du *L^AT_EX*, du SQL ou du code Python.

R Markdown est en quelque sorte un cadre pour travailler en data science. Un même document permet de sauvegarder et exécuter du code ainsi que de produire des rapports mis en forme pouvant contenir une grande diversité d’objets statiques ou dynamiques.

2.4 Du .Rmd au format de sortie



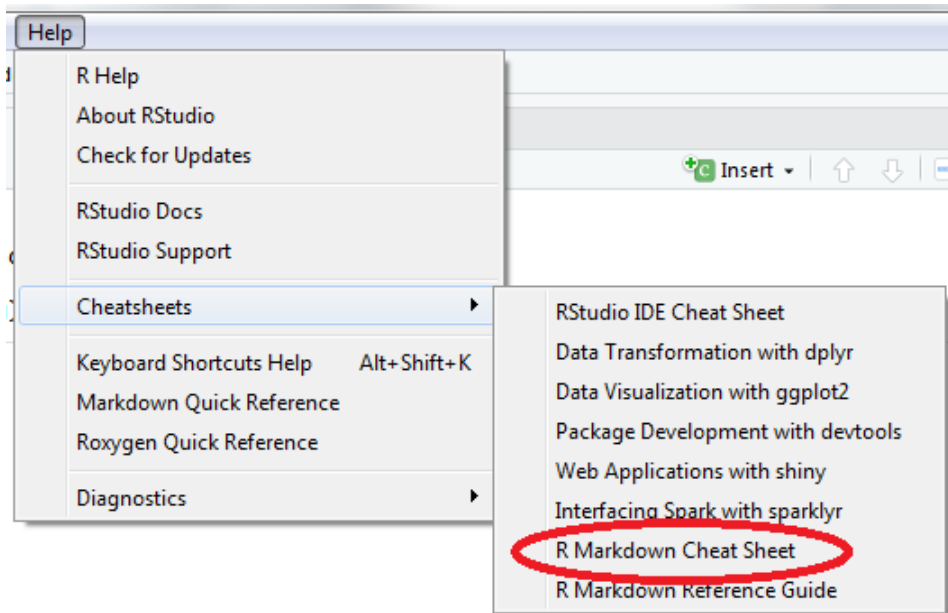
On distingue deux étapes :

- La première, réalisée par le package knitr, interprète le code R présent dans le document R Markdown. Elle aboutit à un fichier Markdown contenant le texte, le code et le résultat de l'exécution du code.
- Puis, le convertisseur de formats Pandoc se charge de produire le document de sortie au format choisi.

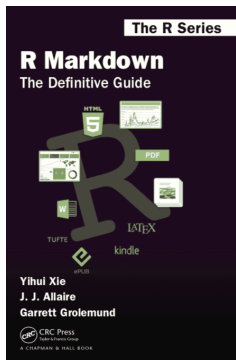
Cela peut paraître compliqué mais en pratique, chacune des fonctions `render` de `knitr` exécute la chaîne du début à la fin.

Chapitre 3 Le fichier R Markdown

La logique et les principaux points à retenir sur Rmarkdown sont rassemblés dans la “cheatsheet” [Rmarkdown](#).



Le [guide définitif de Rmarkdown](#) est un document très complet pour qui veut se lancer ou se perfectionner dans R markdown.



Le [livre de recettes Rmarkdown](#) prolonge le guide Rmarkdown. C’est un florilège d’exemples pratiques répondant à des besoins exprimés sur des forums.



3.1 R Markdown dans RStudio

Pour utiliser R Markdown, il faut que le package `rmarkdown` soit installé :

```
install.packages ("rmarkdown")  
library (rmarkdown)
```

Dans l'environnement de développement intégré RStudio de votre bureau, le développeur dispose d'outils qui simplifient la production d'un fichier R Markdown.

The image shows three panels of RStudio illustrating the workflow for creating an R Markdown document:

- SOURCE EDITOR:** The top-left panel shows the source code of a file named `report.Rmd`. It includes a header with document metadata (title, author, output format), R code chunks for setting up the document and running `summary(cars)`, and a text block. Numbered callouts 1 through 6 point to specific actions: 1. New File, 2. Embed Code, 3. Write Text, 4. Set Output Format(s) and Options, 5. Save and Render, and 6. Share.
- RENDERED OUTPUT:** The top-right panel shows the rendered HTML output (`report.html`). It displays the document title, author name, and the output of the `summary(cars)` function as a table. Callouts point to features like 'find in document', 'publish to', 'reload document', and 'file path to output document'.
- VISUAL EDITOR:** The bottom-left panel shows the visual editor view of the document. It allows for editing the text and code directly. Callouts point to 'insert citations', 'style options', and 'add/edit attributes'.

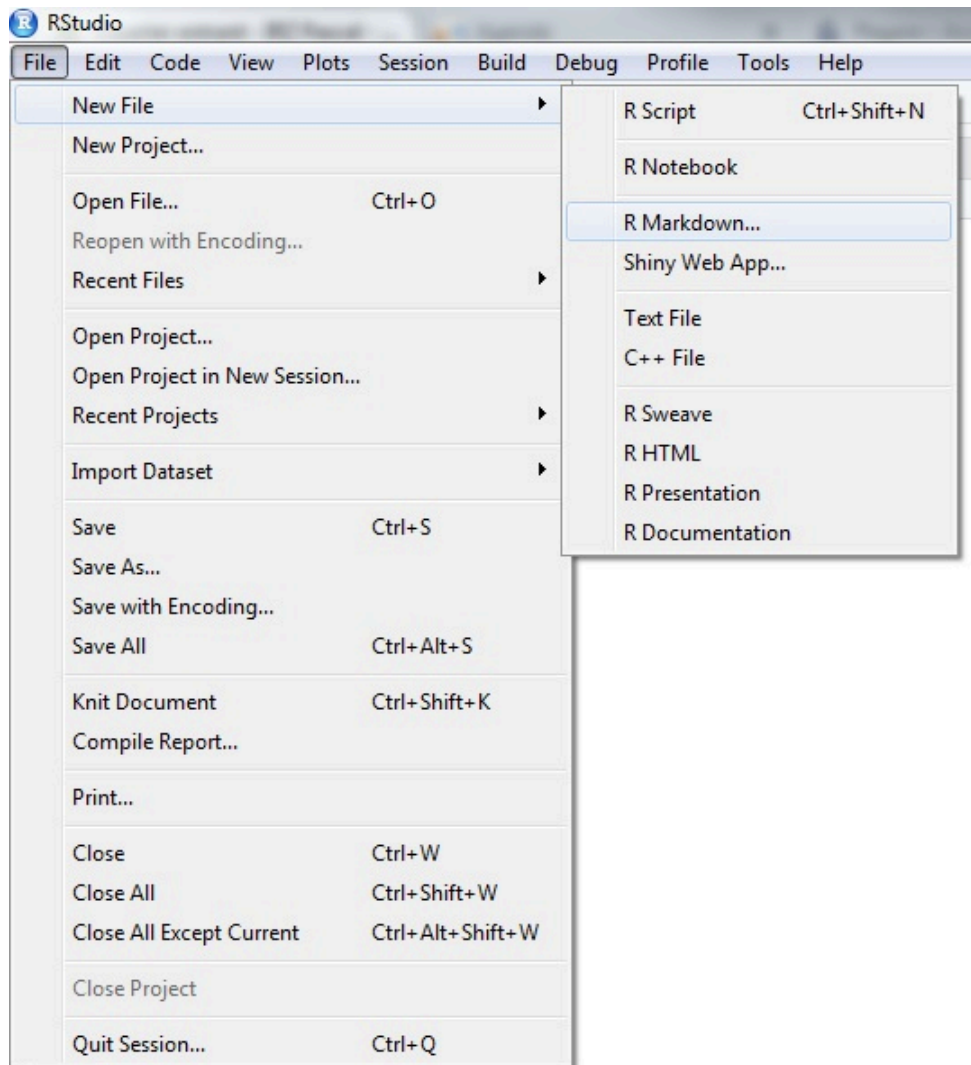
Le travail comprend généralement les étapes suivantes :

1. Ouvrir un nouveau fichier `.Rmd` (pré-rempli avec des exemples ou vierge)
2. Écrire des instructions de traitement des données
3. Compléter le document en ajoutant du texte
4. Adapter l'en-tête
5. Lancer la génération du document
6. Publier le document si besoin est

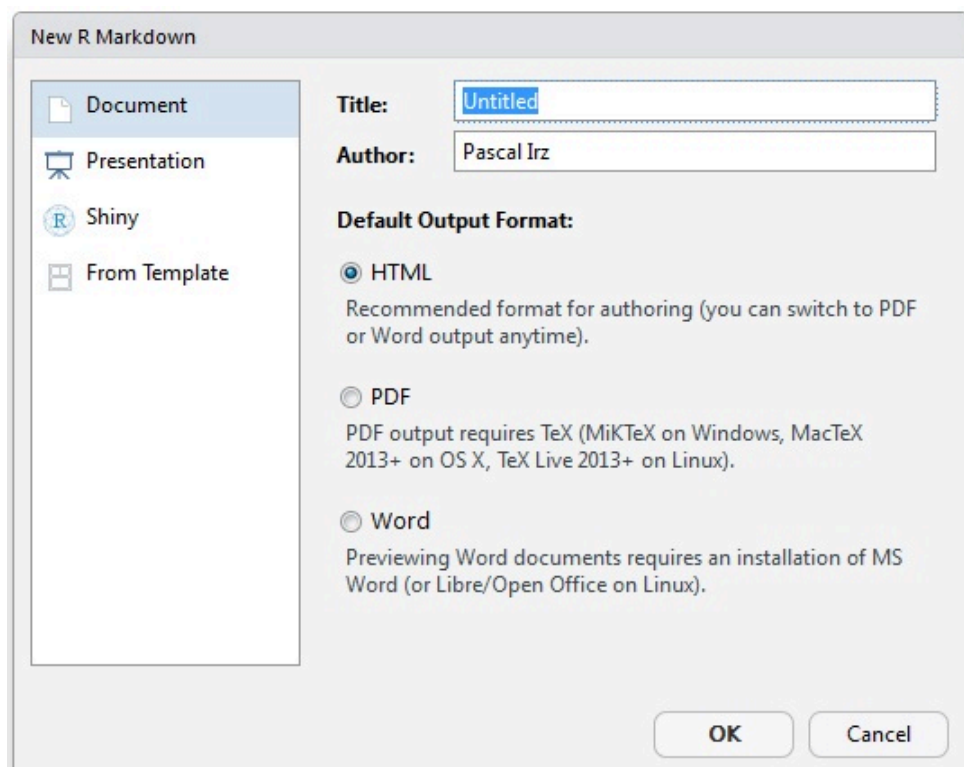
Le travail est généralement itératif et les étapes 2 à 4 peuvent être effectuées dans n'importe quel ordre.

3.1.1 Créer un fichier .Rmd

On crée un nouveau fichier de type Rmarkdown en allant dans le menu File ⇒ New file ⇒ R Markdown...

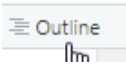


Une pop-up s'ouvre alors pour donner quelques informations afin d'initialiser le fichier qui va être créé en remplissant quelques champs imposés ou fortement recommandés par le processus global.



Pour commencer simplement, on choisit un format de sortie HTML. C'est le format de sortie par défaut, très utilisé. D'autres types de sorties existent, elles seront détaillées par la suite. RStudio crée un fichier contenant des exemples d'éléments structurant un fichier Rmarkdown. L'extension de ce type de fichier est `.Rmd`.

3.1.2 Lancer la génération du document par un 'knit'

Une fois le fichier `.Rmd` complété et enregistré, vous pouvez visualiser la table des matières en cliquant sur l'icône  en haut à droite. Vous pouvez aussi prévisualiser le document en sortie en cliquant sur le bouton "Visual" qui donne accès au mode visuel de l'éditeur de fichier.

Pour générer un document à partir du fichier `.Rmd`, on clique sur le bouton **knit**.

Un fichier (html si vous avez choisi ce format de sortie) portant le même nom que le fichier `.Rmd` est alors créé dans le répertoire de travail (là où vous avez créé votre projet R), et le document généré s'ouvre dans le viewer de R Studio (ou dans une pop-up).

Trucs & Astuces

On peut générer le document en ligne de commande :

```
rmarkdown::render(nom_fichier.Rmd)
```


ou grâce au raccourci `Ctrl + Shift + K`

Le document produit contient l'ensemble des informations du fichier .Rmd, que cela soit le texte (mis en forme), la visualisation du code intégré (si celle-ci est demandée) et ce qui est produit par ce code.

3.2 Les éléments d'un fichier .Rmd

Un fichier R Markdown est constitué de 3 éléments principaux.

Métadonnées YAML	{	1 --- 2 title: "Module 6 - Documents reproductibles avec RMarkdown" 3 author: "Groupe des référents R" 4 date: "19 janvier 2019" 5 output: html_document 6 ---
Chunks		7 8 {r setup, include=FALSE} 9 knitr::opts_chunk\$set(echo = TRUE) 10
Texte		11 12 # Introduction 13 14 ## Le parcours de formation 15 16 Le parcours de formation proposé est structuré en modules de 2 jours chacun : 17 18 - 1 : Socle de base 19 - 2 : Préparer ses données avec le tidyverse 20 - 3 : Statistiques descriptives 21 - 4 : Analyses multivariées

L'**en-tête** contient les métadonnées pour guider la génération du document. Il est suivi du corps du futur document, constitué de **morceaux de code** (les *chunks*) à exécuter et de partie de **textes** à afficher avec des éléments de mise en forme.

Dans les sections suivantes ces composants seront détaillés un par un.

3.2.1 L'en-tête

L'en-tête s'écrit au format **YAML** c'est un format ayant pour objectif de représenter des informations plus élaborées que le simple CSV en gardant cependant une lisibilité presque comparable.

L'en-tête est parfois appelé le Header ou le YAML. Il sert à définir des paramètres comme les informations basiques sur le document ainsi que des choix relatifs au format de sortie (PDF, HTML, DOCX, etc.). Il est délimité par deux séries de `---`.

Pour modifier l'en-tête, attention à bien respecter l'alignement des indentations. Ce sont elles qui indiquent la hiérarchie entre les éléments texte.

C'est dans le YAML que vous allez définir notamment :

- le titre (initialisé lors de la création du fichier)
- les auteurs
- la date de votre document (qui peut être fixe en texte ou appelée en R depuis l'horloge système)
- le format de sortie (initialisé lors de la création du fichier)
- des options
- des paramètres.

```
---
title: "mon_premier_document"
author: "Moi"
date: "`r format(Sys.Date(), '%d %B %Y')`"
output: html_document
---
```

On peut rajouter plusieurs auteurs :

```
---
title: "mon_premier_document"
author:
  - "Moi"
  - "Toi"
date: "`r format(Sys.Date(), '%d %B %Y')`"
output: html_document
---
```

On peut rajouter des options pour un output, ce qui sera vu en détail dans le paragraphe [5](#).

Il est possible de rajouter plusieurs types d'outputs :

```
---  
title: "mon_premier_document"  
author:  
  - "Moi"  
  - "Toi"  
date: "`r format(Sys.Date(), '%d %B %Y')`"  
output:  
  pdf_document: default  
  html_document:  
    toc: true  
    theme: flatly  
---
```

3.2.2 Exercice 1

- Ouvrir Rstudio
- Créer un fichier Rmarkdown, format html
- prévisualiser le document avant toutes modifications
- Se mettre en auteur du document
- Rajouter un theme spécifique
- Cliquer sur *knit* pour compiler le document et identifier les correspondances entre fichier .Rmd et fichier html

Résultat attendu :

Infographie de fromagerie, camemberts interdits !!

Exercice 1

3.2.3 Les éléments texte

Le texte s'écrit en syntaxe Markdown, qui est en fait du PANDOC pour Markdown. Vous trouverez la documentation complète sur le PANDOC pour Markdown [ici](#).

Trucs & Astuces

De manière générale, il est préférable de laisser au moins une ligne blanche entre différents éléments (par exemple entre un titre et le paragraphe).

Cela évite toute confusion lors de la génération du document (tous les processus de génération n'interprètent pas exactement la même chose pour une même syntaxe).

3.2.3.1 Le texte simple

Le texte simple est directement interprété sans besoin de balisage.

3.2.3.2 La mise en forme à la volée

Pour les *caractères en italique*, entourer d'un `*` ou d'un `_` : `*caractères italiques*` .

Pour les **caractères gras**, entourer de deux `**` ou deux `__` : `**caractères gras**` .

Pour les indices et les exposants, entourer des caractères `~indices~` ou `^exposants^` respectivement.

Pour les barrés, entourer des caractères `~~barrés~~` .

Pour un style `code` , entourer des caractères ``code`` .

Pour un bloc de code, entourer des caractères ````` où on commence chaque ligne par au moins 4 espaces.

Pour forcer le retour à la ligne, terminer par un double espace ou un `\` ou sauter une ligne.

3.2.3.2.1 Les titres

Contrairement à la pratique en R, le `#` indique les titres. Par défaut, dans Rmarkdown, ils ne sont pas numérotés.

- `#` pour les titres de niveau 1
- `##` pour les titres de niveau 2
- `###` pour les titres de niveau 3

rend :

Titre de niveau 1

Titre de niveau 2

Titre de niveau 3

Si l'on souhaite qu'une section ne soit pas numérotée, par exemple pour l'introduction ou les annexes, il faut faire suivre son titre de {-}. Exemple : # Annexe A {-}.

3.2.3.2.2 Les listes

Liste à puces

Les listes à puces, sans ordre, commencent par `-` ou `*` ou `+`, en précédant la liste par une ligne vide. On peut créer des listes imbriquées en indentant la sous-liste.

```
- Premier élément
- Deuxième élément
    - Sous-élément 1
        - est-ce bien raisonnable...
        - ...d'imbriquer 3 niveaux ?
    - Sous-élément 2
```

rend :

- Premier élément
- Deuxième élément
 - Sous-élément 1
 - est-ce bien raisonnable...
 - ...d'imbriquer 3 niveaux ?
 - Sous-élément 2

Liste numérotée

- ```
1. Premier élément
2. Deuxième élément
 1) Sous-élément 1
 a- Sous-sous-élément a
 b- Sous-sous-élément b
 1) Sous-élément 2
```

rend :

- ```
1. Premier élément
2. Deuxième élément
    1. Sous-élément 1
        a- Sous-sous-élément a
        b- Sous-sous-élément b
    2. Sous-élément 2
```

Notez qu’une erreur de numérotation sera automatiquement corrigée tant que l’indentation et le style de numérotation sont corrects.

3.2.3.2.3 Les mentions

Les “mentions” ou citations sont pratiques pour mettre en valeur les éléments “à retenir”.

```
> Texte à mettre en mention
```

Rend :

```
| Texte à mettre en mention
```

3.2.3.2.4 Liens et notes de bas de page

Pour faire un **lien** vers un site internet, on met le texte affiché entre `[ ]` et l’adresse http entre `()`

```
[Cliquez sur le lien](https://mtes-mct.github.io/parcours-r/)
```

rend

[Cliquez sur le lien](https://mtes-mct.github.io/parcours-r/)

Pour faire des **liens internes**, on peut utiliser la même syntaxe, après avoir inséré une balise là où on souhaite rediriger le lecteur.

Exemple de placement de la balise :

```
### Commentaires ? {#commentaires}
```

Puis pour faire un lien dessus :

```
[Aller au § Commentaires](#commentaires)
```

La balise `\@ref(commentaires)` crée également un lien vers le paragraphe ‘Commentaires’, mais renvoie son numéro.

texte saisi	résultat après knit
<code>\@ref(commentaires)</code>	3.2.3.3

Les **notes de bas de page** s’écrivent à l’intérieur de `^[]` .

```
...pour approfondir^[on ajoute une note de bas de page] rend : ...pour approfondir1.
```

3.2.3.3 Commentaires

Pour mettre une partie du fichier en commentaires, non traitée, il faut encadrer la partie par `<!-- commentaires -->` .

Pour cela on peut utiliser le raccourci clavier `Ctrl + Shift + C` ou la commande dans le menu “Code”.

3.3 Insérer des images

La syntaxe la plus simple pour insérer une image est la suivante :

```
![Légende de la figure.](chemin/vers/image.png)
```

On peut adapter les dimensions de l’image :

```
![Légende de la figure.](chemin/vers/image.png){ width=50% }
```

Trucs & Astuces

NB : Pour définir les dimensions de l’image, les caractères ‘espace’ ne sont pas autorisés autour du signe égal `=` , ni entre la parenthèse fermante et l’accolade ouvrante. On a `) {` .

On peut aussi utiliser la fonction `knitr::include_graphics` :

```
knitr::include_graphics("assets/img/couleuvre.jpg")
```



Cette dernière méthode, préconisée quand le format de sortie n'est pas du html, permet de mieux contrôler l'affichage de l'image.

Cela se fait à l'intérieur d'un chunk et ça, on le verra au chapitre [4](#)

Attention au répertoire de travail ! Par défaut, dans un script Rmarkdown, le répertoire de travail (donc l'adresse à partir de laquelle commence le chemin de l'image) se situe au niveau du dossier dans lequel est enregistré le fichier .Rmd, même si vous travaillez dans le cadre d'un projet. Se référer à la partie dédiée [1.1.1](#) pour s'en dépatouiller.

3.4 Encodage

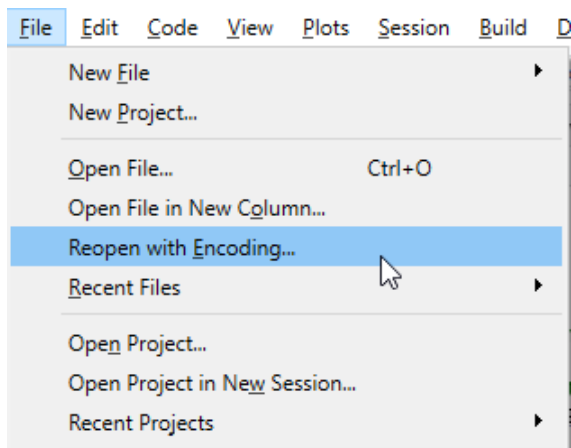
Pour le bon fonctionnement général, tous les fichiers R Markdown doivent être encodés en UTF-8, notamment pour les langages présentant des caractères accentués (comme le français).

Trucs & Astuces

RStudio facilite la gestion de l'encodage des fichiers édités :

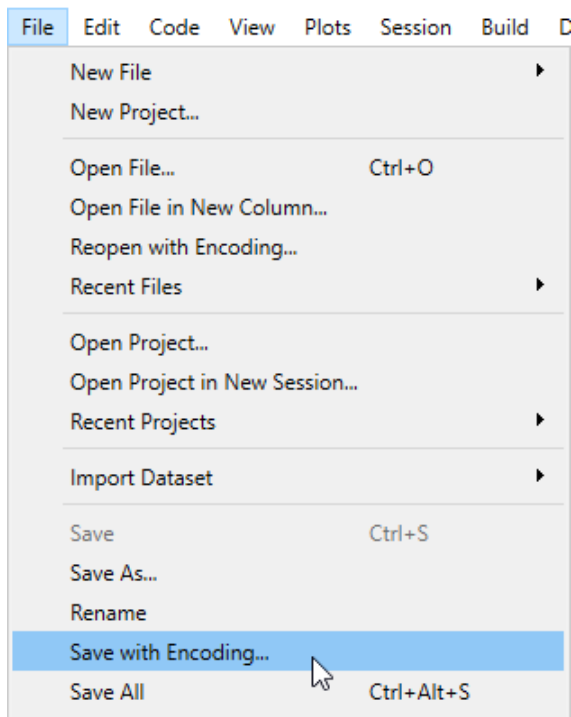
- pour ouvrir un fichier avec l'encodage qui vous convient

Fichier ⇒ Réouvrir avec encodage...



- pour enregistrer un fichier au bon encodage

Fichier ⇒ Sauvegarder avec l'encodage...



Remarquer la case à cocher au niveau de la fenêtre de dialogue “Réouvrir avec encodage...”, sélectionner UTF-8 et cocher l’option : “Définir comme encodage par défaut pour les fichiers sources”, ce sera fait une fois pour toute.

3.5 Exercice 2

Nous allons travailler sur les informations du site www.cheese.com. Nous nous appuyons pour cela sur l'exemple proposé pour l'évènement [tidytuesday](#) du 4 juin 2024.

Le fichier markdown qui sera créé pendant toute cette formation parlera donc de fromage !

- Partir du fichier .Rmd de l'exercice précédent
- Saisir un titre, un paragraphe, une liste d'item, un encart et un lien. N'hésitez pas à jouer avec la mise en forme du texte en syntaxe markdown.
- Appuyer sur `visual` pour avoir un aperçu du rendu final

Résultat attendu :

Infographie de fromagerie, camemberts interdits !!

Exercice 2

Nicolas Torterotot

2026-08-29

La présente infographie se veut être un exemple parfaitement inutile de ce qu'on peut produire à l'aide du format Rmarkdown.

On y présentera :

- les possibilités rédactionnelles et de mise en page pour mettre en valeur le *fromage* ;
- l'intégration d'images, graphiques et cartes pour mieux voir le  ;
- les possibilités de génération automatique à partir d'une série de paramètres pour toujours plus de fromage ;
- un peu d'interactivité pour du fromage 2.0.

1. Au début était le lait

Pour commencer, il nous faut récupérer de la donnée. Nous allons travailler sur les informations du site

1. on ajoute une note de bas de page↩

Chapitre 4 Intégrer du code à exécuter dans les Rmd

4.1 Les *chunks*

4.1.1 La base

Les parties de code R sont contenues dans des blocs, appelés *chunks*.

Lors de la compilation du Rmd, knitr va exécuter le code contenu dans ces blocs et ‘tricoter’ leurs résultats avec le texte saisi en dehors.

Ces chunks commencent et finissent par les balises `````.

C’est dans les chunks que l’on insère le code R que l’on veut voir exécuté dans le rapport.

```
```{r cars, echo=FALSE}
summary(cars)
```
```

On peut créer un nouveau chunk en cliquant sur le bouton :



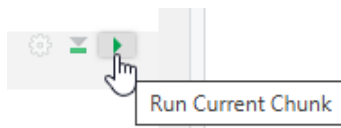
ou grâce au raccourci clavier `Ctrl + Alt + I`.

On peut contrôler le résultat du code en l’exécutant :

- comme dans un script R, en sélectionnant les ligne(s) puis en utilisant le raccourci clavier

`Ctrl + Entrée`,

- ou en utilisant le bouton en triangle vert, situé en haut à droite de chaque chunk



RStudio contient d’autres boutons pour exécuter le code d’un fichier Rmd : ils sont placés dans le menu déroulant ‘Run’ en haut à droite. Ils sont rappelés dans le menu “Code” tout en haut, au niveau de l’entrée ‘Run Region’.

Dans l'environnement de travail (cadran 'Global Environnement', situé en haut à droite généralement dans RStudio), on retrouve les objets créés par R lors des exécutions de chunks précédents.

La commande 'Run All Chunks Above' que l'on peut déclencher en clic bouton ou avec le raccourci

Alt + Ctrl + Maj + P permet d'être certains de travailler sur les objets tels que créés précédemment par le programme. Lors du knitr, R utilise au démarrage un environnement R vierge.

4.1.2 Le nom des chunks

Les chunks peuvent être nommés (avec des caractères alphanumériques minuscules et majuscules et des tirets -).

Dans l'exemple précédent, le nom du chunk est 'cars'.

Nommer les chunks est utile, principalement pour se repérer dans le document.

Un chunk nommé "setup" a des propriétés spéciales. Il sera systématiquement exécuté avant les autres chunk du document, ça en fait le lieu tout indiqué pour placer les commandes de chargement des packages (`library(dplyr)` ...) ou de lecture des données qui seront exploitées par la suite. Il est généralement placé en début de document, en dessous de l'en-tête.

4.1.3 Les options des *chunks*

Au début de chaque chunk se trouve une accolade contenant la lettre r : elle indique à knitr que le code à exécuter est en langage R, cf [4.3](#).

C'est dans cette accolade, après la lettre r, que les options vont pouvoir être passées.

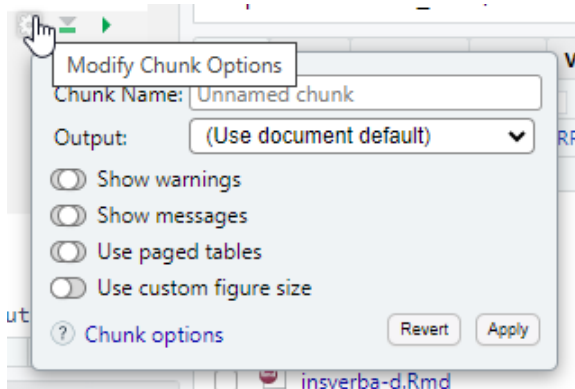
Elles permettent de contrôler finement ce qui est produit par le chunk, pour choisir de faire apparaître, ou non, le code dans le rapport dynamique, ainsi que les résultats, ou encore pour définir la taille des plots.

Chaque chunk peut recevoir une ou des options. Voici quelques exemples utilisés fréquemment :

- `eval = TRUE` : Le chunk est exécuté.
- `include = FALSE` : Le code contenu dans le chunk est exécuté sans que soient affichés ni le code ni son résultat.
Cela rend ses résultats utilisables par d'autres chunks.
- `echo = FALSE` : permet de ne pas afficher le code dans le rendu. En revanche, le résultat est affiché.
- `message = FALSE` : empêche l'affichage des messages d'information générés par le code.
- `warning = FALSE` : empêche l'affichage des messages d'alerte générés par le code.
- `error = FALSE` : bloque la compilation du document si une erreur est renvoyée.

- `fig.cap = "..."` : ajoute une légende (un 'caption') aux graphiques.
- `fig.align = "..."` : aligne les graphiques (choix : `left` , `right` ou `center`).
- `fig.height = 6, fig.width = 8` : permet de modifier les dimensions de la figure (en pouces).

Pour éviter de se rappeler de tout cela, les options les plus fréquentes sont paramétrables en clic/bouton depuis la roue crantée à côté du triangle vert d'exécution du chunk :



Il y a plus de 50 options possibles pour un chunk, on trouve l'ensemble des possibilités [dans la documentation de {knitr}](#) , sur lequel `{Rmarkdown}` se base pour gérer les chunks.

Il est possible de définir globalement les options de chunks, pour éviter de les saisir à chaque fois, grâce à la fonction `knitr::opts_chunk$set()` . Elles s'appliqueront alors à chacun des chunks du document Rmd, sauf spécification contraire au niveau des options d'un chunk particulier.

```
```${r} setup, include=FALSE}
knitr::opts_chunk$set(
 message = FALSE,
 warning = FALSE
)
```
```

Dans cet exemple, on demande à ne pas faire apparaître les messages ou les warnings parfois générés lors de l'exécution de code, et qui viendraient polluer le rapport mis en forme.

4.1.4 Intégrer des visualisations R

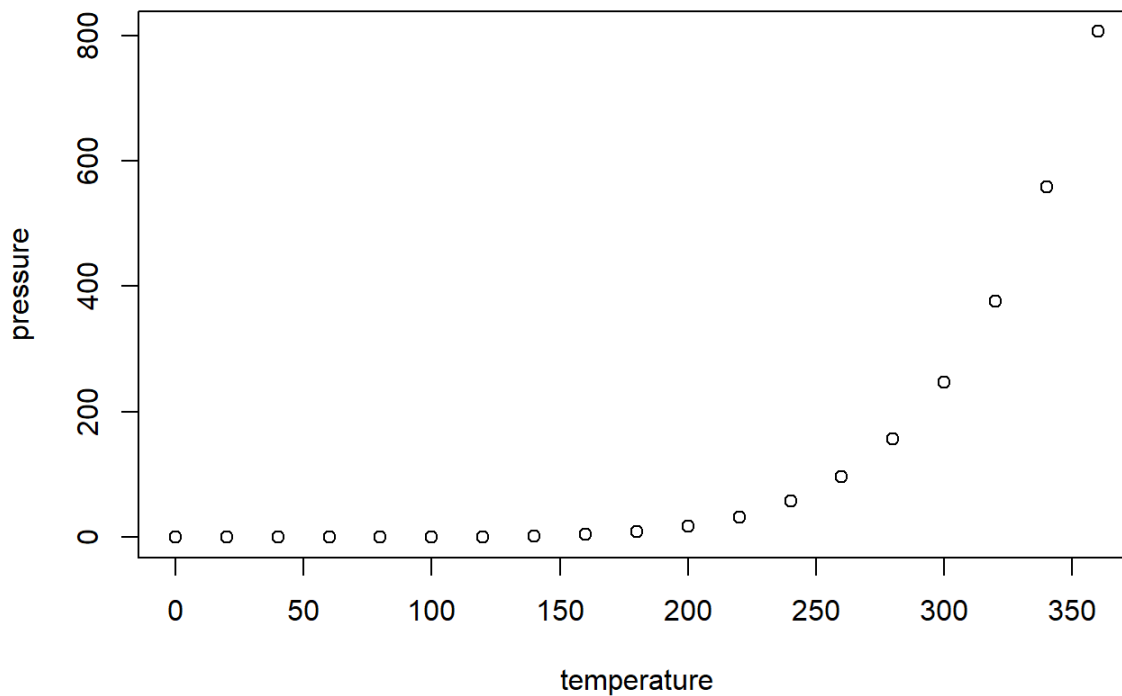
A l'intérieur du chunk, de nombreuses choses peuvent être faites, comme traiter des données, produire une table, des graphiques ou du texte.

Pour cela, on utilise différentes fonctions R comme `plot` ou `kable` dans un chunk.

On peut par exemple produire un **graphique** :

```
```{r, echo=FALSE}
plot(pressure)
```
```

ce qui affiche le graphique dans le document, directement après le chunk :



Ou des **données** non mises en forme :

```
```{r, echo=FALSE}
summary(cars)
```
```

```
##      speed      dist
##  Min.   : 4.0    Min.   :  2.00
## 1st Qu.:12.0    1st Qu.: 26.00
## Median :15.0    Median : 36.00
## Mean   :15.4    Mean    : 42.98
## 3rd Qu.:19.0    3rd Qu.: 56.00
## Max.   :25.0    Max.    :120.00
```

Ou des **tableaux** :

```
```{r, echo=FALSE}
knitr::kable(iris[1:5,], caption = 'Le titre du tableau')
```
```

qui rend

Table 4.1: Le titre du tableau

| Sepal.Length | Sepal.Width | Petal.Length | Petal.Width | Species |
|--------------|-------------|--------------|-------------|---------|
| 5.1 | 3.5 | 1.4 | 0.2 | setosa |
| 4.9 | 3.0 | 1.4 | 0.2 | setosa |
| 4.7 | 3.2 | 1.3 | 0.2 | setosa |
| 4.6 | 3.1 | 1.5 | 0.2 | setosa |
| 5.0 | 3.6 | 1.4 | 0.2 | setosa |

4.2 Les insertions de code dans le texte : le code inline

Enfin, il est possible d'**insérer la valeur d'objets R** (variable, liste, résultat de calcul simple...) dans du texte. Pour cela il faut inclure l'objet R entre ``r ...code...``. C'est très utile pour commenter des résultats sans saisir en "dur" des valeurs issues d'un calcul qui dépend de données d'entrée, susceptibles d'être modifiées.

Par exemple le code suivant:

```
```{r, echo=FALSE}
numero <- 6
```

Je suis actuellement en train de me former au module `r numero`.
```

permet d'afficher dans le document :

Je suis actuellement en train de me former au module 6.

4.3 Autres langages

Les portions de code d'un document .Rmd peuvent être formulées dans d'autres langages grâce au package knitr.

Le [guide Rmarkdown liste les langages possibles](#) et présente comment les mobiliser.

Pour retrouver cette liste :

```
names(knitr::knit_engines$get())
```

Pour utiliser un autre langage, on change l'appel entre accolades en haut du chunk, par exemple :

```
```{python}
x = 'hello, python world!'
print(x.split(' '))
```
```

Cela permet d'employer des langages faits pour l'interactivité évaluée côté client comme le JavaScript et s'affranchir d'un déploiement sur serveur R.

Attention, si le format Rmarkdown (fichier avec extension .Rmd) supporte d'autres langages, l'inverse n'est pas vrai. Vous ne pourrez pas nativement ouvrir un fichier .Rmd dans votre console python par exemple.

Pour une cross compatibilité complète, il faudra utiliser le format Quarto (extension .qmd) qui sera présenté dans le chapitre [10.1](#).

4.3.0.1 Cas particulier du SQL

Pour utiliser du SQL, il faut impérativement ajouter un paramètre de connexion à l'aide d'une variable préalablement fixée dans un chunk R :


```

```{r, include = FALSE}
library(RPostgres)
library(DBI)

conn <- dbConnect(drv = Postgres(),
 dbname = "ma_base",
 host = "adresse_serveur",
 port = "port",
 user = "utilisateur",
 password = "mdp")

```

```

ATTENTION : il est très dangereux de noter ses informations de connexion en clair dans un script qui pourra être partagé ou versionné sur Git.

Vous pouvez les stocker en variables d'environnement (à faire une fois sur son poste, dans la console R, ne pas enregistrer dans un script) avec la fonction `Sys.setenv` :

```

Sys.setenv("user_pg" = "votre nom utilisateur",
           "pwd_pg" = "votre mot de passe")

```

Puis les appeler dans vos scripts ou documents reproductibles avec la fonction `Sys.getenv` :

```

conn <- dbConnect(drv = Postgres(),
                  dbname = "ma_base",
                  host = "adresse_serveur",
                  port = "port",
                  user = Sys.getenv("user_pg"),
                  password = Sys.getenv("pwd_pg"))

```

Par suite, une fois la connexion établie, vous pouvez exécuter du code SQL en spécifiant la connexion.

```

```{sql connection=conn}
SELECT * FROM mon_schema.matable
```

```

Cette instruction renvoie le résultat de la requête SQL (les données sélectionnées).

Pour pouvoir stocker ce résultat afin de le mobiliser ultérieurement, il faut ajouter le nom de l'objet entre "" dans les options du chunk au niveau `output.var=` :

```
```{sql, connection=conn, output.var="resultat"}
SELECT * FROM mon_schema.matable
```
```

Dans cet exemple, les données sélectionnées seront stockées dans `resultat` .

N'oubliez pas de fermer la connexion avec `dbDisconnect(conn)` une fois les données récupérées, votre gestionnaire de BdD vous remerciera !

4.4 Exercice 3

- Poursuivre le fichier .Rmd de l'exercice précédent
- Créer un chunk pour définir des options générales, il ne doit pas être visible dans le document final
- Créer un chunk pour charger les packages nécessaires, il ne doit pas être visible dans le document final
- Créer un nouveau chapitre qui aura pour objet la présentation de quelques observations du jeu de données
- Ajouter un chunk pour lire les données fromage . Le code ne doit pas s'afficher dans le document final.

Pour récupérer les données tidyuesday : `load("extdata/data_fromage_tidyuesday.RData")` depuis votre projet d'exercices

créé avec savoirR ou `tidyuesdayR::tt_load('2024-06-04')`

(prérequis : avoir installé le package tidyuesdayR et configuré votre .Renviron avec un token github d'accès personnel,

ie créé une variable `GITHUB_PAT`)

```
load("extdata/data_fromage_tidyuesday.RData")
```

- Ajouter une ligne de texte qui indique le nombre de fromages disponibles dans la dataframe (cette valeur doit être calculée automatiquement)
- Créer une variable dans cette dataframe pour indiquer si l'origine des fromages (colonne `country`) est française ou non
- Ajouter un chunk créant un graphique en barres qui indique le nombre de fromage par origine (FR ou non-FR). Le graphique devra apparaître dans le document final, pas le code pour le créer.

- Ajouter une image d'illustration en tête d'article
- Générer le document

Résultat attendu :

Infographie de fromagerie, camemberts interdits !!

Exercice 3

Nicolas Torterotot

2026-08-29



© Olga Yastremska / iStock images

La présente infographie se veut être un exemple parfaitement inutile de ce qu'on peut produire à l'aide du format Rmarkdown.

On y présentera :

- les possibilités rédactionnelles et de mise en page pour mettre en valeur le **fromage** ;
- l'intégration d'images, graphiques et cartes pour mieux voir le 🧀 ;
- les possibilités de génération automatique à partir d'une série de paramètres pour toujours plus de fromage ;
- un peu d'interactivité pour du fromage 2.0.

1. Au début était le lait

Pour commencer, il nous faut récupérer de la donnée. Nous allons travailler sur les informations du site www.cheese.com (<https://www.cheese.com/>). Nous nous appuyons pour cela sur l'exemple proposé pour l'évènement tidyuesday du 4 juin 2024 (<https://github.com/rfordatascience/tidyuesday/blob/main/data/2024/2024-06-04/readme.md>).

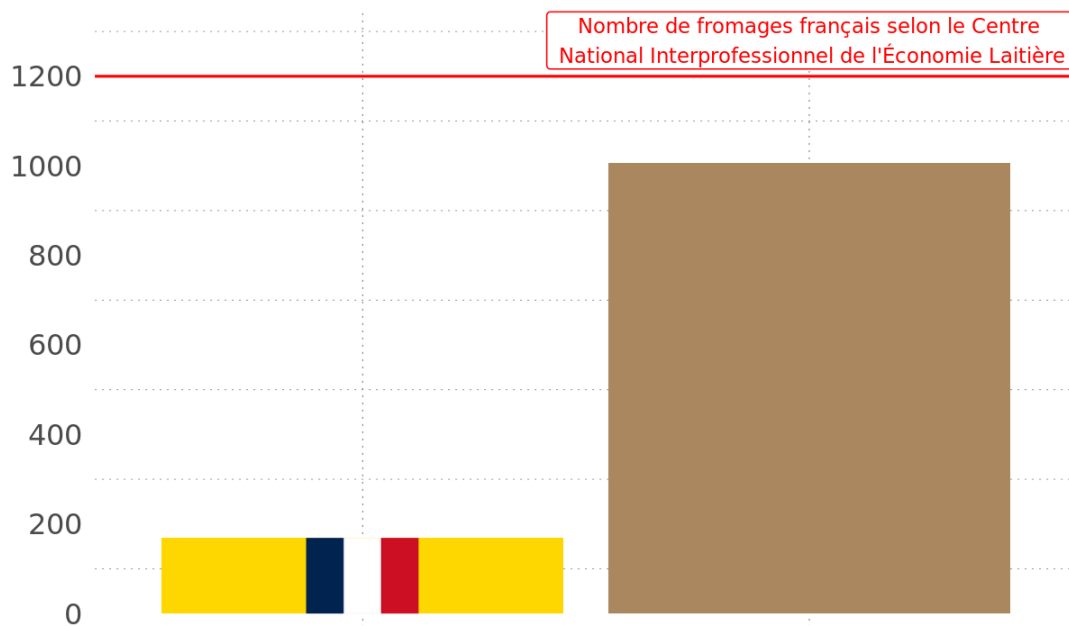
2. Premières observations

2. PREMIERES OBSERVATIONS

Notre jeu de données contient des informations sur **1187** fromages.

On constate tout de suite une grande infamie, **un biais terrible, un crime de lèse fromajesté** :

Nombre de fromages référencés sur cheese.com par origine



Chapitre 5 Les formats de sortie

Lors de la création du fichier .Rmd, vous avez dû choisir le format de sortie par défaut du document qui sera généré. Ce choix se retrouve dans l'option `output` de l'en-tête.

Il est possible de modifier le format du document généré, soit en modifiant l'option de l'en-tête ou celle de la fonction knit (via la flèche vers le bas du bouton knit ou via les options de la ligne de code).

Il existe deux types de format dans le package `{rmarkdown}` : documents et présentations.

Tous les formats de sorties inclus dans le package `{rmarkdown}` sont listés ci-dessous:

- `beamer_presentation`
- `context_document`
- `github_document`
- `html_document`
- `html_vignette`
- `ioslides_presentation`
- `latex_document`
- `md_document`
- `odt_document`
- `pdf_document`
- `powerpoint_presentation`
- `rtf_document`
- `slidy_presentation`
- `word_document`

Chaque format a son propre lot d'options, qui sont documentées au niveau de la fonction associée (par exemple `?rmarkdown::html_document`).

Comme souvent avec R, il existe de nombreuses extensions qui complètent les possibilités offertes par `{rmarkdown}` . Elles sont listées sur <https://rmarkdown.rstudio.com/formats.html/> et, pour les plus connues d'entre elles, présentées dans le [guide Rmarkdown au niveaux des chapitres 5 à 10](#).

Trucs & Astuces

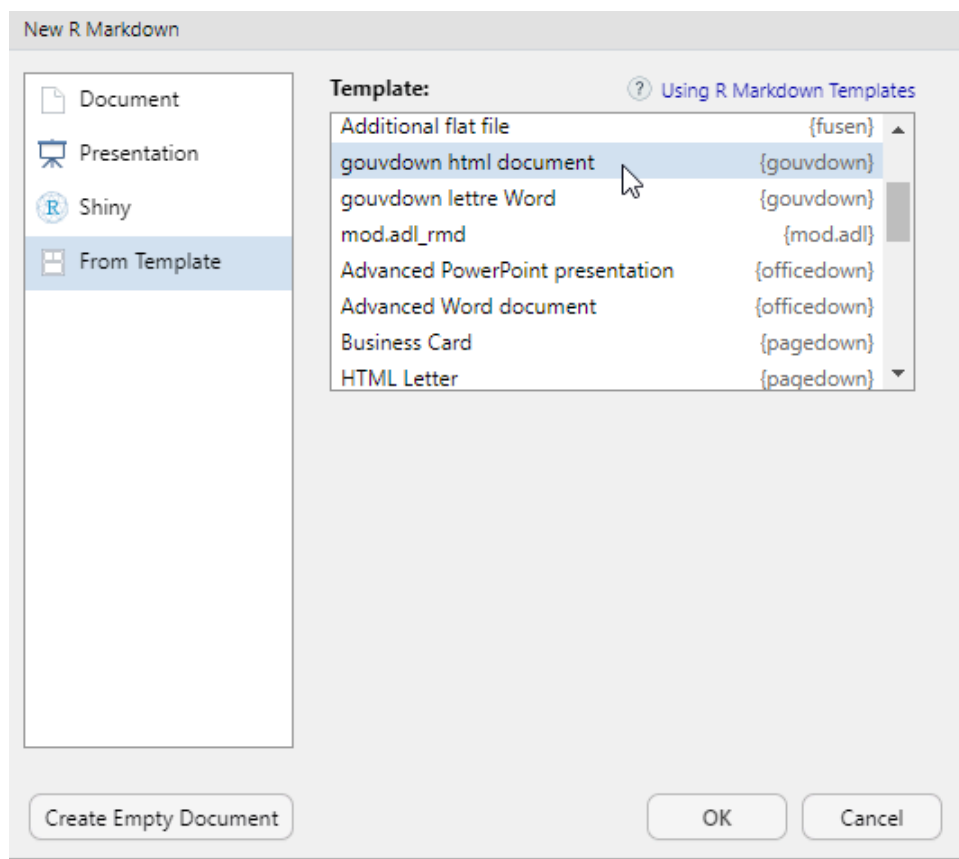
Attention : quand on inclut du code R dans un document, il faut s'assurer de la compatibilité entre les visualisations produites et le(s) format(s) de document(s) choisi(s). Généralement, quand on perfectionne une visualisation pour un meilleur affichage au format HTML, on perd la possibilité de produire une sortie vers un autre type de format.

Lorsqu'on fait appel à un package externe pour définir notre format de sortie, il est nécessaire de le préciser dans l'entête yaml, au niveau de `output:` .

Par exemple, pour utiliser le format 'html_gouv' du package `{gouvdown}` qui sert mettre les productions R à la marque Etat, voici ce qu'il faut indiquer :

```
---  
output: gouvdown::html_gouv  
---
```

La plupart des packages proposant des modèles de documents Rmd les rendent également accessibles en clic bouton depuis le menu File ⇒ New file ⇒ R Markdown...



Ces menus apparaissent une fois le package qui les contient installé.

Nous allons vous présenter les formats les plus utiles.

5.1 HTML avec `html_document`

A l'origine, Markdown a été conçu pour générer du HTML, c'est ce format qui a le plus de possibilités parmi tous les formats. C'est aussi le plus compatible avec les différents packages R de production de visualisation.

Pour obtenir un document HTML basique, il faut mettre l'option `output:html_document` dans l'en-tête.

Voici les options utilisées fréquemment propres au format `html_document` :

- `toc: true` permet d'ajouter une table des matières (table of contents en anglais) à notre document.
- `toc_depth: 2` permet de définir le niveau de titre le plus bas à mettre dans la table des matières (par défaut 3).
- `toc_float: true` permet de rendre la table des matières flottante. Elle sera systématiquement visible, même si on défile le document. Ce paramètre accepte des options.
- `number_sections: true` permet de numérotter les titres.
- `theme: flatly` permet de changer le thème du document (tiré de la librairie [Bootstrap](#)).
- `fig_width: 7` et `fig_height: 5` permettent de définir par défaut la largeur et la hauteur des figures.
- `fig_caption: true` permet de définir que les figures contiennent une légende.
- `css: adresse/fichier/css` permet d'appliquer un css particulier au document HTML rendu (cf. [Personnaliser le css](#))
- `self_contained: TRUE/FALSE` pour générer ou non un fichier HTML autoportant (cf [Option self_contained](#))

L'ensemble des options est accessible depuis l'aide de la fonction `rmarkdown::html_document` (? `rmarkdown::html_document`)

```
---
title: "mon_premier_document"
author:
  - "Moi"
  - "Toi"
date: "31/10/2025"
output:
  html_document:
    toc: true
    theme: flatly
---
```


5.1.1 Personnaliser le css

L'un des avantages du format HTML est la possibilité d'appliquer un style personnalisé à l'aide d'un fichier CSS. Ces documents, qui servent à mettre en forme tous les éléments d'une page HTML par la définition de styles à appliquer, utilisent une syntaxe spécifique mais relativement simple :

- il faut cibler un élément de la page à partir de sa classe (pour styliser tous les éléments de ce type) ou de son identifiant (pour styliser uniquement cet élément là), on peut assigner une classe spécifique à un élément ;
- il faut spécifier le(s) paramètre(s) qu'on veut styliser (la police, la couleur, l'alignement, l'espacement, etc.) ;
- il faut fixer une valeur pour ce(s) paramètre(s) ;
- les paramètres sont rassemblés entre accolades, séparés de leurs valeurs par un double point, séparés entre eux par un point virgule.

Par exemple, si on veut que tous les titres de niveau H1 soient en gras rouge on écrira :

```
```{css, eval = F}

h1 {
 font_weight: bold;
 color: red;
}

```
```

Il est possible d'intégrer directement cette portion de css dans un chunk du document mais la bonne pratique est plutôt de créer un document à part (extension .css ou .scss, c'est une option proposée par Rstudio dans File > New File) et de le référencer dans le yaml, en reprenant l'exemple précédent :

```
---
title: "mon_premier_document"
author:
  - "Moi"
  - "Toi"
date: "31/10/2022"
output:
  html_document:
    toc: true
    theme: flatly
    css: "mon_style.css"
---
```

À noter : le style fixé par un chunk au sein du document l'emportera sur le style d'un fichier css à part qui l'emportera sur le style fixé par un thème qui l'emportera sur le style de base de l'objet. La dernière information de style lue prévaudra sur les précédentes.

De nombreuses ressources en ligne sont disponibles pour apprendre les bases du css, tous les paramètres qui peuvent être fixés et leurs modalités, par exemple [celles de la fondation Mozilla](#).

5.1.2 Option `self_contained`

Par défaut, R Markdown génère des fichiers HTML auto-portants, sans dépendances externes. Tout est intégré dans le fichier .html : feuilles de style, librairies js, des images, graphiques, polices, vidéos... On peut ainsi partager ou publier le fichier comme un document bureautique, sans devoir l'accompagner d'un dossier "html_files" et on est certain de ce qui est restitué.

Mais cela peut alourdir le document et freiner son affichage web.

Si on préfère conserver les dépendances dans des fichiers externes, pour faciliter l'affichage web, il faut spécifier `self_contained: false` dans les options de l'entête. Par exemple :

```
---
title: "mon_premier_document"
output:
  html_document:
    self_contained: false
---
```

Cela a par ailleurs l'avantage de placer les figures dans le dossier des dépendances du html et de pouvoir les mobiliser facilement dans d'autres contextes.

5.1.3 Affichages sur plusieurs colonnes

Cela se fait très bien en format PDF, c'est plus compliqué en HTML. Vous pourrez avoir besoin de définir un document en plusieurs colonnes, avec par exemple une illustration sur une moitié de la page, un texte sur l'autre.

Pour cela vous allez devoir séparer vos colonnes dans des *“div”*.

Dans markdown, des div commencent par `::: {}` et finissent par `:::` .

Pour ensuite que le document aligne ces blocs l'un à côté de l'autre, vous devrez utiliser la propriété css `display: flex;` dans une `div` englobante.

5.1.4 Accessibilité visuelle des documents html produits

Certains choix peuvent fortement améliorer (ou dégrader) l'accessibilité pour les personnes porteuses d'un handicap visuel ou utilisant un lecteur d'écran. Voici les bonnes pratiques principales, adaptées aux html produits avec RMarkdown. Pour une présentation complète, se référer au [Référentiel général d'amélioration de l'accessibilité \(RGAA\)](#).

5.1.4.1 Structurer le document avec des titres hiérarchisés et intégrer une table des matières

Les lecteurs d'écran utilisent la structure du document pour naviguer.

1. Utiliser les niveaux de titres Markdown (`#`, `##`, `###`)
2. Respecter la hiérarchie (ne pas passer de `#` à `####`)
3. Eviter de simuler un titre avec du gras
4. Ajouter une table des matières avec `toc: true` dans l'entête yaml.

Grâce à cela, on crée un plan du document et les lecteurs d'écran peuvent naviguer de titre en titre.

5.1.4.2 Mettre un nom de substitution aux images

Les images doivent avoir un texte alternatif (alt text).

```
![Histogramme de la distribution des âges](histogramme.png)
```

Conseils pour rédiger le texte alternatif : décrire le message principal et les tendances importantes.

5.1.4.3 Perception des couleurs

Veiller quand c'est possible, à ne pas transmettre d'informations uniquement par la couleur. Cela rendrait le document illisible pour les personnes atteinte de daltonisme ou ayant une faible perception des contrastes.

Dans les graphiques R, il faut chercher à utiliser une palette de couleur en redondance avec les formes ou les annotations et choisir des palettes de couleurs accessibles, comme `viridis` :

```
ggplot(data, aes(x, y, color = groupe, shape = groupe)) +  
  geom_point() +  
  scale_color_viridis_d()
```

5.1.4.4 Rendre les tableaux lisibles

Pour proposer des tableaux accessibles, on limite leur largeur et on utilise des entêtes claires. Si le tableau est complexe, il convient d'ajouter un titre et un texte explicatif.

5.2 Sorties PDF

5.2.1 Configuration préalable pour l'usage de `pdf_document`

Pour générer des documents PDF avec `output: pdf_document`, il est nécessaire d'avoir installé LaTeX. LaTeX est un logiciel externe qui vient en complément de R pour produire des PDF. Si LaTeX n'est pas déjà installé par ailleurs, nous conseillons de passer par le package R `{tinytex}`, qui guide l'utilisateur de R pour installer sur son ordinateur une distribution LaTeX légère, portable et facile à maintenir.

Le code suivant permet d'installer la package R `{tinytex}` :

```
install.packages('tinytex')
```

On installe ensuite la distribution LaTeX sur le PC avec la commande `install_tinytex()` de `{tinytex}`. On peut spécifier le dossier d'installation souhaité (choisir un dossier sur lequel on a les droits d'écriture :-)) :

```
# Exemple de choix d'adresse pour l'installation sur un PC
install_path <- file.path(Sys.getenv('LOCALAPPDATA'), 'tinytex')
tinytex::install_tinytex(dir = install_path)
```

En cas de difficultés, il est parfois nécessaire d'indiquer à R où se trouve ce nouvel exécutable.²

5.2.2 Spécificités du format pdf_document

Pour obtenir une conversion du Rmd en un document PDF, il faut mettre l'option `output:pdf_document` dans l'en-tête. Tous les paramètres de ce format de sortie sont accessibles depuis l'aide de la fonction `rmarkdown::pdf_document` (`?rmarkdown::pdf_document`)

Voici les options utilisées fréquemment:

- `toc: true` permet d'ajouter une table des matières (table of contents en anglais) à notre document.
- `toc_depth: 2` permet de définir le niveau de titre le plus bas à mettre dans la table des matières (par défaut 2).
- `number_sections: true` permet de numérotter les titres.
- `fig_width: 7` et `fig_height: 5` permettent de définir par défaut la largeur et la hauteur des figures (par défaut 6.5x4.5).
- `fig_caption: true` permet de définir que les figures contiennent une légende (par défaut `true`).

Il est important de noter qu'en LaTeX, les figures sont flottantes par défaut.

Même si on crée un graphique dans un bout de code présent sur la 1ère page, celui-ci peut finalement apparaître sur la page suivante. LaTeX a tendance à faire apparaître les figures au début ou à la fin des pages. Il est conseillé de ne traiter le positionnement des figures qu'à la fin du process, une fois la totalité du contenu écrite pour des ajustements pérennes.

Pour cela il faudra utiliser les options de positionnement dans les chunks (par exemple `fig.pos="h"`). Les [valeurs possibles pour fig.pos](#) sont détaillées dans la documentation de LaTeX.

| fig.pos | effet |
|---------|--|
| h | Place l'image au plus près possible de l'endroit où elle est définie dans le texte source. |
| t | Positionne en haut de page. |
| b | Positionne en bas de page. |
| p | Positionne sur une page spéciale dédiée aux figures. |
| ! | Remplace les paramètres par défaut LaTeX pour déterminer les « bonnes » positions. |
| H | Place l'image à l'endroit précis où elle est définie dans le Rmd source, équivalent à h! . |

Vous pouvez indiquer plusieurs valeurs dans le paramètre `fig.pos` . Par exemple, si vous indiquez `"ht"` , LaTeX tentera d'abord de positionner la figure à cet endroit. Si cela s'avère impossible (par exemple, faute d'espace suffisant), la figure apparaîtra en haut de la page. Il est recommandé d'utiliser plusieurs paramètres de positionnement afin d'éviter tout résultat inattendu.

Trucs & Astuces Si vous souhaitez accéder à plus d'options de mise en forme de tableau, le package `kableExtra` contient des fonctions compatibles avec les formats HTML et PDF. Cependant cela reste compliqué de mettre en forme des tableaux complexes si vous souhaitez plusieurs formats en sortie. Nous vous conseillons d'adapter la mise en forme pour chaque format dans deux chunks distincts ou de trouver une nouvelle façon de représenter la donnée. Une autre option consiste dans un 2e temps à imprimer le html généré avec une imprimante PDF.

5.2.3 {Pagedown}

Le package `{pagedown}` avec sa fonction `chrome_print()` permet de produire un pdf à partir d'un Rmd conçu pour du html, sans besoin de LaTeX.

La fonction se repose sur le moteur d'impression du navigateur chrome :

- plus besoin de coder différemment les sorties visuelles selon le format (html ou PDF),
- plus besoin d'installer LaTeX (mais besoin de chrome ou chromium),
- et on conserve la mise en forme css lors du passage au format pdf !



Le package propose également un format de sortie html `pagedown::html_paged` qui facilite la pagination du document pdf imprimé.

```
---  
output:  
  pagedown::html_paged:  
    toc: true  
    number_sections: false  
---
```

Son fonctionnement est détaillé au niveau du chapitre [Pour aller plus loin sur Pagedown](#).

5.3 Multiplier les formats de sorties

Lorsqu'on utilise des sorties de tableaux ou d'illustrations basiques, ou lorsqu'on on a pris soin de ne sélectionner que des instructions compatibles avec tous les formats de sortie, ou on peut multiplier les formats de sorties d'un même document R Markdown en empilant dans l'entête la liste des formats de sorties souhaités.

```
---  
output:  
  gouvdown::html_gouv:  
    toc:true  
  pdf_document:  
    toc:true  
  odt_document: default  
---
```

Pour comparer les options permises par chaque type de sorties, se référer à la [feuille de triche RMarkdown](#), également accessible depuis le menu Help / Cheat Sheets / R Markdown Cheat Sheet qui comporte un comparatif.

5.4 Exercice 4

En repartant du fichier .Rmd de l'exercice précédent :

- Modifier les options pour avoir un sommaire flottant affichant un seul niveau de titre
- Changer le thème utilisé pour la sortie HTML

- Générer le document
- Ajouter un format de sortie à l'entête pour qu'il génère un document PDF et un document odt en plus du HTML. Rappel : il est important de vérifier la compatibilité des éléments produits par le code avec ce nouveau format.

Résultat attendu :

Infographie de fromagerie, camemberts interdits !!

Exercice 4

Nicolas Torterotot

2026-08-29

- 1. Au début était le lait
- 2. Premières observations



© Olga Yastremska / iStock images

La présente infographie se veut être un exemple parfaitement inutile de ce qu'on peut produire à l'aide du format Rmarkdown.

On y présentera :

- les possibilités rédactionnelles et de mise en page pour mettre en valeur le **fromage** ;
- l'intégration d'images, graphiques et cartes pour mieux voir le 🧀 ;
- les possibilités de génération automatique à partir d'une série de paramètres pour

toujours plus de fromage ;

- un peu d'interactivité pour du fromage 2.0.

1. Au début était le lait

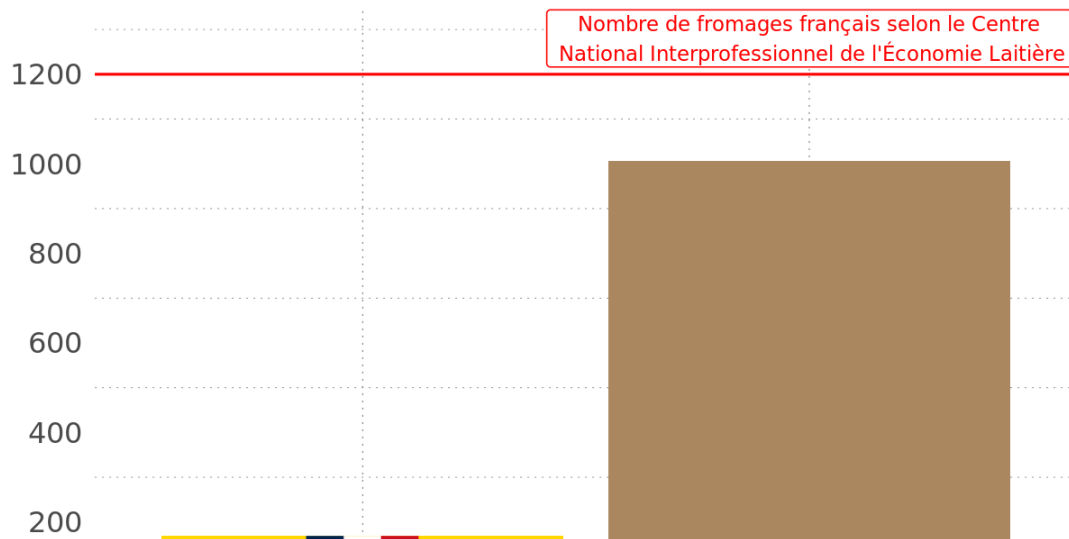
Pour commencer, il nous faut récupérer de la donnée. Nous allons travailler sur les informations du site www.cheese.com (<https://www.cheese.com/>). Nous nous appuyons pour cela sur l'exemple proposé pour l'évènement tidyuesday du 4 juin 2024 (<https://github.com/rfordatascience/tidyuesday/blob/main/data/2024/2024-06-04/readme.md>).

2. Premières observations

Notre jeu de données contient des informations sur **1187 fromages**.

On constate tout de suite une grande infamie, **un biais terrible, un crime de lèse fromajeté** :

Nombre de fromages référencés sur cheese.com par origine



2. Pour cela, il faut modifier le fichier de variables d'environnement de R (le fameux .Renviron) présent dans notre Home. (Au besoin `usethis::edit_r_environ()` permet d'éditer facilement ce fichier.) Il faut y ajouter les lignes de code suivantes :

```
TINYTEX_HOME="${LOCALAPPDATA}\\tinytex\\bin\\win32"
```

```
PATH="${TINYTEX_HOME};${PATH}"
```



Chapitre 6 Paramétrer un rapport

Un des nombreux avantages de R Markdown est la possibilité de reproduire des analyses très facilement en actualisant une partie du travail ou en changeant un des entrants du document. Utiliser des paramètres permet d'aller encore plus loin pour créer un document qui peut être réutilisé pour plusieurs scénarios. On peut ainsi créer des documents pour des territoires ou des années différents, faire tourner une analyse en changeant une des hypothèses ou changer le comportement de `knitr` selon les cas rencontrés.

6.1 Ajouter une entrée `params` dans l'entête yaml

Les paramètres sont spécifiés dans l'en-tête avec l'option `params` dans laquelle plusieurs paramètres, et leur valeur par défaut, sont listés, un par ligne.

Les paramètres peuvent être de type `character`, `numeric`, `integer` et `logical` mais aussi des expressions R tant qu'elles sont précédées de `!r`. L'en-tête, et donc le code pouvant y être présent, est exécuté avant le reste du code donc il est nécessaire d'explicitement les packages utilisés.

Par exemple:

```
---
title: "mon_premier_document"
author: "Moi"
date: "31/10/2025"
output: html_document
params:
  annee: 2024
  region: Bretagne
  date: !r lubricate::today()
---
```

Une fois définis dans l'en-tête, ces paramètres sont accessibles depuis le fichier `.Rmd` (texte ou code) mais aussi depuis la console. Ils sont stockés dans une liste nommée `params`.

6.2 Exemple d'utilisation des paramètres

Le code suivant montre quelques exemples d'utilisation des paramètres, dans les options de chunks, ou dans la production des visualisations :

```
---  
title: "mon_premier_document"  
author: "Moi"  
date: "31/10/2022"  
output: html_document  
params:  
  espece: setosa  
  printcode: TRUE  
---
```

```
```${r, setup, include=FALSE}  
knitr::opts_chunk$set(
 message = FALSE,
 warning = FALSE,
 echo = params$printcode
)
```
```

Les résultats affichés ci-dessus concernent l'espèce `\${params\$espece}`.

```
```${r}  

print(params)

summary(iris)

extrait <- iris %>%
 filter(Species==params$espece)

```
```

Il existe 3 façons de générer un document avec des paramètres :

- utiliser le bouton **knit**, ce qui prend les valeur par défaut des paramètres ;
- utiliser l'interface RStudio en sélectionnant l'option **Knit with Parameters** du bouton **knit**. Cela ouvre une nouvelle fenêtre demandant de choisir les valeurs des paramètres indiqués dans l'en-tête ;
- utiliser la fonction `rmarkdown::render()` . Sans autre précision cette fonction utilisera les paramètres par défaut définis dans l'en-tête. On peut aussi définir de nouvelles valeurs en utilisant l'option `params =` . Cela donne par exemple :

```
```{r, echo=FALSE}
rmarkdown::render("mon_premier_document.Rmd",
 params = list(espece = "versicolor", printcode = FALSE))
```
```

Cette dernière façon de faire, via la fonction `rmarkdown::render()` permet d'automatiser encore plus les choses en générant autant de documents que de valeurs différentes d'un paramètre. En effet, dans un script .R, il est possible de créer une fonction ayant en entrée les paramètres que l'on veut modifier plusieurs fois mais aussi le nom du document généré.

6.3 Comment compiler un rapport pour une liste de valeurs ?

En reprenant l'exemple précédent, on peut produire la sortie d'un rapport pour toutes les espèces :

```

```{r, echo=FALSE}
Listes des especes contenu dans le dataset iris
liste_especes <- iris %>%
 pull(Species) %>%
 unique() %>%
 as.character()

une fonction qui génère le rapport en fonction de l'espece
generer_rapport_par_espece <- function(nom_espece) {
 rmarkdown::render("mon_premier_document.Rmd",
 params = list(espece = nom_espece, printcode = FALSE),
 output_file = paste0("doc-", nom_espece, ".html"))
}

Application de la fonction à toutes les espèces grâce à lapply
lapply(X = liste_especes, FUN = generer_rapport_par_espece)
```

```

6.4 Paramétrer le titre du rapport

On peut utiliser du code R pour définir notre titre. La seule contrainte est de déplacer notre balise de titre après la définition des éléments dont il dépend.

```

---
author: "Moi"
date: "`r format(Sys.Date(), '%d %B %Y')`"
output: html_document
params:
  espece: setosa
  printcode: TRUE
title: "Rapport iris - Variété `r params$espece`"
---

```

Rapport iris - Variété setosa

Moi

24 novembre 2025

Les résultats affichés ci-dessus concernent l'espèce setosa.

```
print(params)
```

```
## $espece  
## [1] "setosa"
```

On peut insérer le titre dans une balise yml ailleurs que dans l'entête placée en début de document si besoin de construire le titre avec des calculs R.

```
---  
title: "`r mon_objet_R_contenant_le_titre`"  
---
```

6.5 Exercice 5

- Créer un nouveau .Rmd
- Ajouter comme paramètre dans le YAML le pays (country) en choisissant "France" comme valeur par défaut
- Ajouter un titre pour qu'il dépende de ce paramètre
- Filtrer la dataframe en fonction de ce paramètre
- Ajouter un graphique construit à partir de cette dataframe filtrée. Son titre peut également dépendre du paramètre country. (voir la [page dédiée à la réalisation de graphique avec ggplot dans le module 5 du parcours R](#), ou réutiliser le code ci-dessous)

Par exemple, on peut regrouper les fromages par type simplifié et représenter le pourcentage de matières grasses par type. C'est l'occasion de découvrir ou re-découvrir certaines nouvelles fonctions de dplyr apparues en 2026 :

```

data_fromage_1 <- data_fromage %>%
  filter_out(when_any(is.na(fat_content),
                      is.na(type))) %>%
  mutate(type = replace_when(type,
                              str_detect(type, 'soft') ~ 'molle',
                              str_detect(type, 'firm') ~ 'ferme',
                              str_detect(type, 'hard') ~ 'dure')) %>%
  mutate(fat_content = as.numeric(str_extract(fat_content, "^\\d+(\\.\\d+)?"))) %>%
  filter(country==params$country)

ggplot(data_fromage_1) +
  geom_violin(aes(x = type, y = fat_content, fill = type)) +
  scale_fill_manual(values = c("dure" = "#EEDC82", "ferme" = "#FFFACD", "molle" = "#FFFAF0")) +
  theme_gouv(plot_title_size = 12,
             subtitle_size = 12) +
  labs(title = "% de matière grasse par type de pâte de fromage",
       subtitle = params$country,
       x = "Type de pâte",
       y = "% de matière grasse") +
  theme(legend.position = 'none')

```

- Générer ce HTML à l'aide du bouton knit
- Créer dans un nouveau script une fonction qui génère le HTML à partir d'une liste de pays
- Appliquer cette fonction à une liste restreinte de pays (par exemple France, Italy, United States)

Résultat attendu :

La texture ne fait pas le gras - France

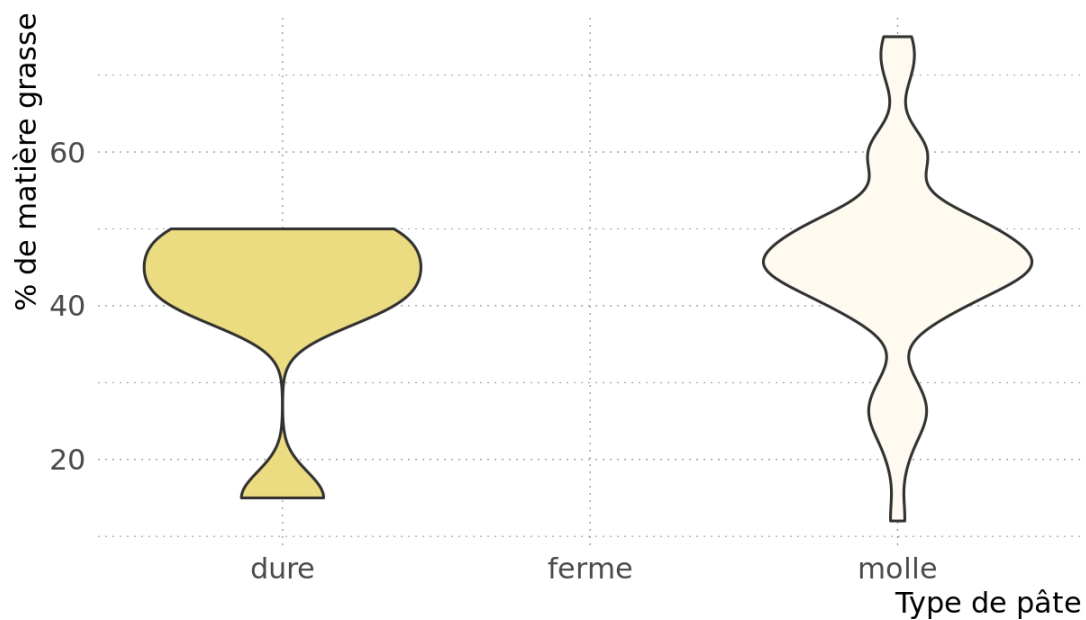
Exercice 5

Nicolas Torterotot

2026-08-29

% de matière grasse par type de pâte de fromage

France



exécution pour plusieurs pays

Chapitre 7 De la page au livre, le package

`{bookdown}`

Lorsqu'une publication comporte un volume important d'éléments, il est très utile d'en organiser le contenu en plusieurs parties.

Le package `{bookdown}` est là pour nous aider à structurer une publication en plusieurs fichiers Rmd.

La publication devient un projet RStudio à part entière.

La séparation d'un document en plusieurs pages (plusieurs fichiers Rmd) a pour avantages d'autoriser l'envoi de liens vers chacun des chapitres et de réduire le temps d'affichage car chaque page est moins lourde prise séparément.

Des fonctionnalités de mise en forme supplémentaires sont ajoutées, telles que les références croisées et la numérotation des figures, des équations et des tableaux.

Les documents peuvent être exportés dans une gamme de formats adaptés à la publication (PDF, epub, HTML).



Trucs & Astuces

Comme pour les fichiers Rmd simples :

- l'export au format pdf nécessite une distribution LaTeX (cf chap [5.2.1](#)).
- et `{pagedown}` peut être mobilisé pour imprimer un ensemble de pages HTML au format pdf, l'assemblage pdf se fait avec `qpdf::pdf_combine()` .

7.1 Comment ça marche ?

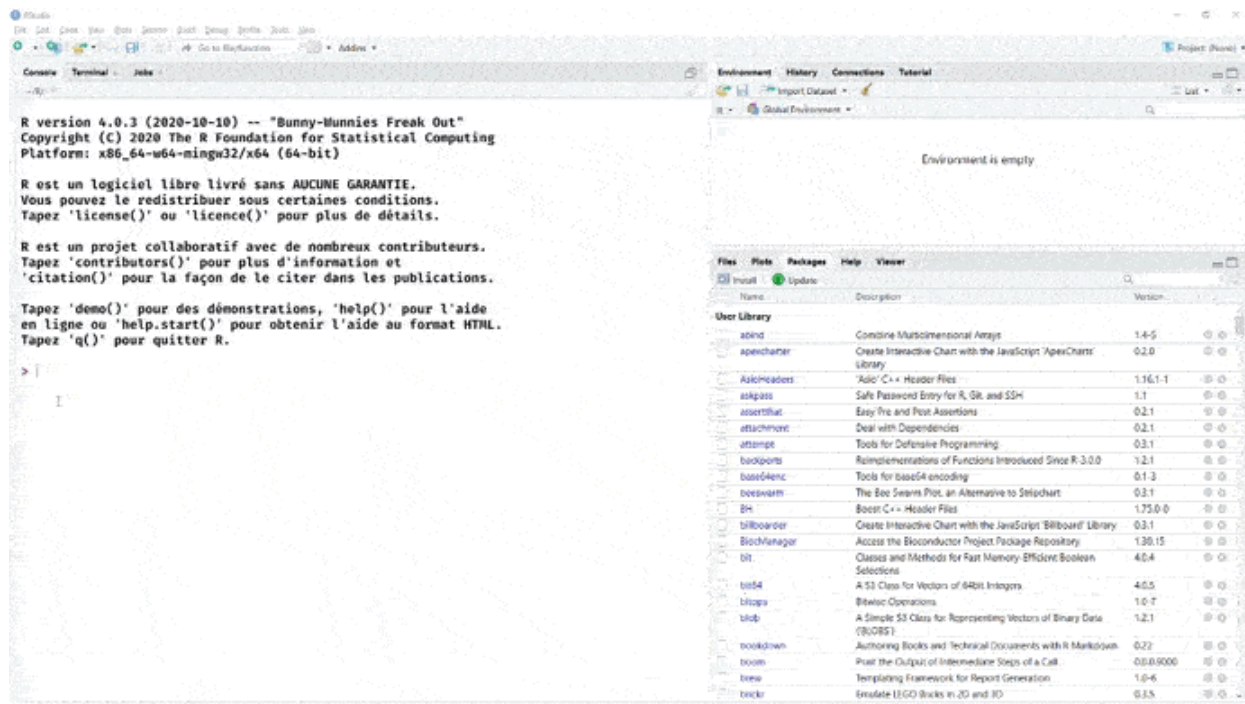
Installer bookdown depuis le CRAN

```
install.packages('bookdown')
```

Depuis Rstudio, cela vous apporte un nouveau type de projet, accessible depuis `File -> New Project -> New Directory -> Book Project using bookdown`.

Une fois celui-ci créé, vous pouvez cliquer sur `Build book / All format` dans l'onglet `Build` de Rstudio. Cela vous compilera le document aux formats html, pdf et epub³ et ouvrira la version html.

A l'instar du package `{rmarkdown}`, qui fournit la fonction `rmarkdown::render(input = "mon_fichier.Rmd")`, `{bookdown}` propose la fonction `bookdown::render_book(input = "mon_dossier")` pour compiler l'ensemble des fichiers `.Rmd` d'un dossier.



D'un point de vue informatique, la fonction `bookdown::render_book` commence par assembler le contenu de tous les documents `Rmd` dans un seul fichier `.Rmd` temporaire, puis compile ce fichier `Rmd` temporaire. En conséquence, les objets R créés dans un chapitre peuvent être mobilisés au chapitre suivant, ou encore, les librairies (packages) n'ont pas besoin d'être appelées dans chacun des chapitres `.Rmd` avec `library()`. Il est conseillé de les appeler toutes au niveau du fichier `index.Rmd`, dans le 1er chunk. En cas d'exécution manuelle d'un chunk, il faudra en revanche exécuter manuellement ceux des pages précédentes dont il dépend.

Les fichiers créés sont enregistrés dans le dossier `_book`. On retrouve la publication compilée en ouvrant le fichier `index.html` dans le navigateur.

Le document par défaut produit présente les principales fonctionnalités de `bookdown`.

7.2 Contenu du projet par défaut

Quand vous créez un projet bookdown, vous avez dans votre projet automatiquement les fichiers suivants :

- `index.Rmd` , le seul fichier Rmarkdown qui contient comme usuellement un yaml en en-tête. C'est le premier chapitre de votre livre et sa page d'accueil.
- `01-intro.Rmd` à `06-references.Rmd` des fichiers rmarkdown correspondant aux chapitres de vos livres. La structure classique d'un rmarkdown est un document Rmd par chapitre, qui seront ensuite pour la version html votre premier niveau de navigation. Chaque fichier commence par le titre du chapitre.
- `_bookdown.yml` un fichier de configuration de votre document bookdown.
- `_output.yml` un fichier de configuration des formats de sortie de votre document (pdf, html...).
- `book.bib` un fichier de bibliographie au format [BibTeX](#).
- `preamble.tex` et `style.css` des fichiers de configuration de l'apparence de votre document pour sa version pdf (réalisé en LaTeX) et sa version html (réalisé en css).

```
directory/
├─ index.Rmd
├─ 01-intro.Rmd
├─ 02-literature.Rmd
├─ 03-method.Rmd
├─ 04-application.Rmd
├─ 05-summary.Rmd
├─ 06-references.Rmd
├─ _bookdown.yml
├─ _output.yml
├─ book.bib
├─ preamble.tex
├─ README.md
└─ style.css
```

7.3 Chaque fichier Rmd correspond à un chapitre.

Il est recommandé de faire commencer chacun d'eux par un titre de niveau 1, pour s'y retrouver dans le projet.

On peut, si besoin, regrouper les chapitres au sein de parties. Pour commencer une partie, ajouter une première ligne `# (PART) Nom de votre partie {-}` devant la ligne de titre de niveau 1 contenant le nom du chapitre, exemple

```
# (PART) Partie I {-}      --> fichier index.Rmd
# Chapitre un

# Chapitre deux           --> fichier 2e_chap.Rmd

# Chapitre trois          --> fichier 3e_chap.Rmd

# (PART) Partie II {-}    --> fichier 4e_chap.Rmd
# Chapitre quatre
```

Ensuite ce qui a été vu précédemment reste applicable : le code s'écrit dans des chunks, le texte partout ailleurs. Il faut relancer la construction (compilation) pour se rendre compte de l'évolution du contenu.

Des chapitres en trop, ou des chapitres à rajouter ? Il suffit de supprimer ou de créer les fichiers `Rmarkdown` correspondant.

Ils seront agencés selon leur ordre alphabétique, donc pour contrôler l'assemblage des chapitres, une pratique courante est de numéroter les fichiers.

L'en-tête yaml du rapport se trouve dans le fichier `index.Rmd` ⁴. Il contient des options spécifiques au format bookdown, notamment sur la gestion de la bibliographie.

```
title: "A Minimal Book Example"
author: "Yihui Xie"
date: "2026-09-02"
site: bookdown::bookdown_site --> type de sorties site html
documentclass: book
bibliography: [book.bib, packages.bib]
biblio-style: apalike
link-citations: yes
description: "This is a minimal example of using the bookdown package to write a book. The output
```

7.4 La structure du book : le fichier `_bookdown.yml`

Le fichier `_bookdown.yml` permet de spécifier des options de configuration supplémentaires pour construire le livre.

Par exemple :

- changer le nom du fichier (`book_filename`)
- franciser le préfixe devant le numéro du chapitre (`chapter_name`) ou le préfixe des tableau et des graphiques (`fig` et `tab`)
- changer l'ordre de fusion des fichiers (`rmd_files`)

```
book_filename: "mon_premier_bookdown"
delete_merged_file: true
language:
  ui:
    chapter_name: "Chaptitre "
  label:
    fig: "Graphique "
    tab: "Tableau "
rmd_files: ["index.Rmd", "01-intro.Rmd", "03-parts.Rmd"]
```

7.5 Le paramétrage des formats de sortie : le fichier

`_output.yml`

Le fichier `_output.yml` est utilisé pour spécifier les type de format de sortie (pdf, html...) et les options relatives à ces formats.

Pour le format html, c'est là par exemple que vous pouvez spécifier entre autre :

- la feuille de style css à utiliser
- les textes inscrits en haut et en bas du menu de navigation
- le style de css à appliquer
- les options de partage sur les réseaux sociaux que vous voulez (si vous en voulez, vous pouvez aussi tous les désactiver avec l'option `sharing` ci contre)

```
bookdown::gitbook:
css: style.css
config:
  toc:
    before: |
      <li><a href="."/>Mon premier bookdown</a></li>
    after: |
      <li>Mon premier bookdown</li>
download: ["pdf", "epub"]
sharing: no
info: no
```

7.6 Pour des rendus stylés

Le package bookdown propose de multiples [modèles de sortie](#) :

- gitbook
- pdf_book
- epub_book
- bs4_book
- bs4_book_theme
- html_chapters
- html_book
- tufte_html_book

Comme toujours avec R, de nombreux packages étendent ces possibilités.

Notamment, le package gouvdown propose un template bookdown `gitbook_gouv`, dérivé du format gitbook, accessible depuis le menu des nouveaux projets R, pour des book conformes à la marque État.

Comme pour les formats de sortie Rmarkdown, il faut avoir en tête que les visualisations optimisées pour un rendu web (HTML), ne seront pas compatibles avec l'export pdf.

7.7 Exercice 6

Expérimenter bookdown

4. Attention, si le projet contient plusieurs fichiers Rmd présentant un en-tête yaml, le dernier écrase les précédents.↩

Chapitre 8 Du Rmarkdown interactif

Quand notre produit de sortie est un pdf, on va vouloir générer des graphiques et cartes qui ont le comportement d'une image simple. C'est ce que nous avons fait depuis le début de ce cours. Mais de plus en plus de nos projets sont destinés à une publication web et un certain niveau d'interactivité est souhaité. La considération principale à prendre en compte est l'environnement de déploiement. Si mon document est hébergé sur un serveur R, j'ai la possibilité de lui faire exécuter du code R "en direct" en fonction des actions de l'utilisateur. Dans le cas contraire, il faudra utiliser de l'interactivité en javascript. Ce langage est interprété et exécuté directement par le navigateur du client (celui qui consulte la page). Mais pas d'inquiétude, vous pouvez continuer à coder uniquement en R, le javascript est caché derrière.

8.1 Interactivité simple, les visuels s'animent

Lorsqu'on choisit le HTML comme format de sortie, on peut utiliser toutes les possibilités d'animation des cartes et graphes vus dans [le module 5 - "Valoriser ses données avec R"](#).

Vous pouvez intégrer des cartes leaflet, des cartes et graphiques ggiraph, des datatables et bien d'autres widgets qui offrent des possibilités d'animations au survol, d'infobulles, de zoom etc. Ces widgets ne nécessitent pas de déploiement sur un serveur R pour fonctionner. Ils sont exécutés par le navigateur client.

On parle ici d'interactivité simple car il s'agit uniquement d'effets visuels dans le comportement des widgets mais il n'y a aucune modification des données dépendante de l'utilisateur ni aucune opération côté serveur.

8.2 Interactivité avancée, les inputs utilisateur

Dans certains cas, on souhaite que l'utilisateur puisse directement influencer les visuels, en filtrant une sous-partie du jeu de données par exemple. Il existe une solution qui permet cela tout en restant dans le cadre d'une exécution par le navigateur client : [le package Crosstalk](#).

Le fonctionnement de base est très simple, on transforme notre objet (la dataframe qui contient les données affichées) en un objet partagé (shared object en anglais) :

```
library(crosstalk)

dataframe_partagee <- SharedData$new(dataframe)
```

Regardez comme cela est noté dans l'environnement de travail.

On peut spécifier un argument qui indique le nom d'une colonne de valeurs toutes différentes permettant d'identifier les lignes de manière unique. On peut voir ça comme une clé primaire en gestion de base de données. Par défaut Crosstalk utilise le numéro de ligne.

```
dataframe_partagee <- SharedData$new(dataframe, ~identifiant)
```

À partir de là, on peut utiliser cet objet partagé à la fois dans un widget de filtre, dans un tableau et dans une carte dynamique pour que l'utilisateur puisse filtrer ses objets à afficher.

```
library(crosstalk)
library(leaflet)
library(DT)

dataframe_partagee <- SharedData$new(dataframe, ~modalites)

filter_select(id = "mon_filtre", label = "Choisissez une modalité", sharedData = dataframe_partagee)

datatable(dataframe_partagee, ...)

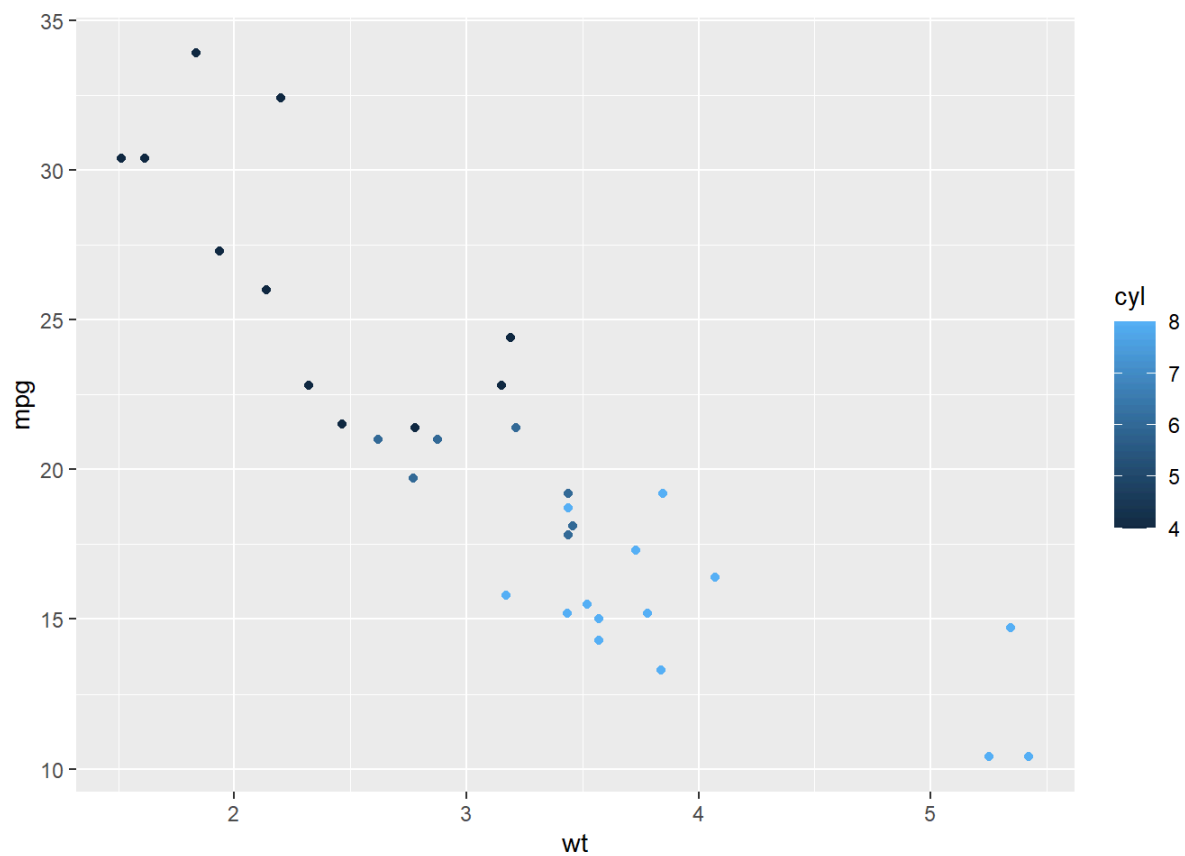
leaflet(dataframe_partagee) %>%
  addCircleMarkers(...)
```

Attention, pour le moment, leaflet ne permet l'utilisation d'une dataframe partagée que pour une représentation ponctuelle. Il n'est pas possible de filtrer des objets polygonaux de cette manière.

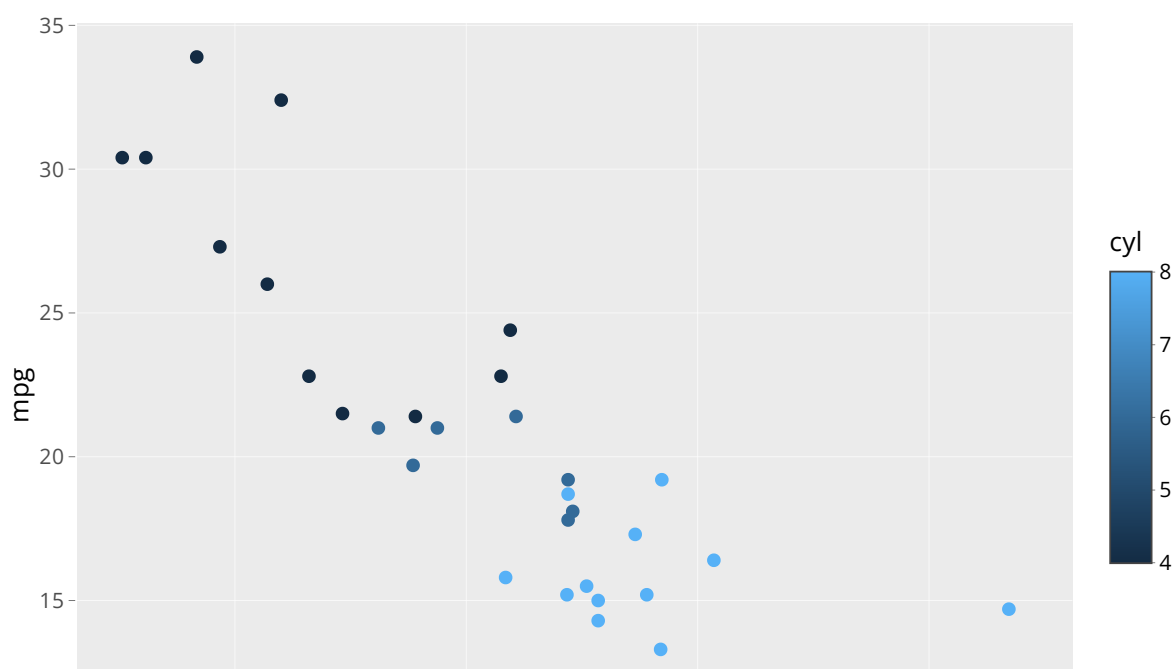
8.3 Exemple reproductible

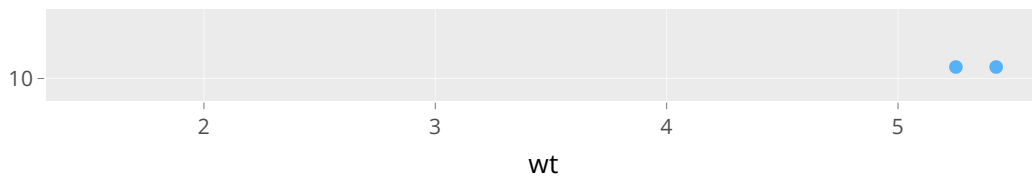
```
library(ggplot2)
library(plotly)
library(crosstalk)
```

```
plot <- ggplot(mtcars, aes(x = wt, y = mpg, color = cyl))+  
  geom_point()  
plot
```



```
ggplotly(plot)
```



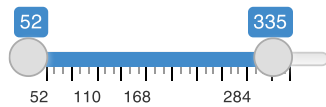


```
shared_mtcars <- SharedData$new(mtcars)
plot <- ggplot(shared_mtcars, aes(x = wt, y = mpg, color = cyl)) +
  geom_point()
plot2 <- ggplot(shared_mtcars, aes(x = hp, y = qsec, color = cyl)) + geom_point()
bscols(widths = c(3, 5, 4),
  list(
    filter_checkbox("cyl", "Nb de cylindres", shared_mtcars, ~cyl, inline = TRUE),
    filter_slider("hp", "Puissance (cv)", shared_mtcars, ~hp, width = "100%"),
    filter_select("auto", "Automatique", shared_mtcars, ~ifelse(am == 0, "Oui", "Non"))
  ),
  ggplotly(plot, width=250, height=250),
  ggplotly(plot2, width=250, height=250)
)
```

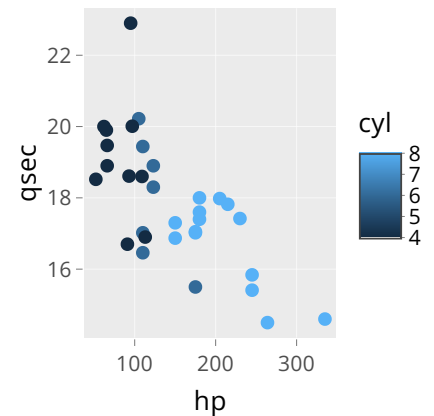
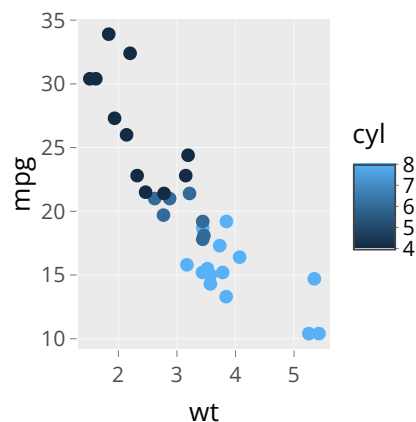
Nb de cylindres

☐ 4 ☐ 6 ☐ 8

Puissance (cv)



Automatique



8.4 Exercice 7 (facultatif)

- Repartir du .Rmd de l'exercice 4 au format HTML
- Ajouter un nouveau chapitre relatif à des visualisations interactives (si besoin : voir la [partie interactivité des visualisations du module 5 du parcours](#))
- Créer un widget HTML interactif : un graphique ggiraph, un tableau datatable, une carte leaflet...
- Créer une dataframe partagée à l'aide du package crosstalk
- Utiliser cette dataframe partagée pour créer un filtre crosstalk sur la colonne country
- Créer un widget interactif à partir de cette dataframe partagée

Résultat attendu :

Infographie de fromagerie, camemberts interdits !!

Exercice 7

Nicolas Torterotot

2026-08-29



© Olga Yastremska / iStock images

La présente infographie se veut être un exemple parfaitement inutile de ce qu'on peut produire à l'aide du format Rmarkdown.

On y présentera :

- les possibilités rédactionnelles et de mise en page pour mettre en valeur le **fromage** ;
- l'intégration d'images, graphiques et cartes pour mieux voir le 🧀 ;
- les possibilités de génération automatique à partir d'une série de paramètres pour toujours plus de fromage ;
- un peu d'interactivité pour du fromage 2.0.

1. Au début était le lait

Pour commencer, il nous faut récupérer de la donnée. Nous allons travailler sur les informations du site www.cheese.com (<https://www.cheese.com/>). Nous nous appuyons pour cela sur l'exemple proposé pour l'évènement tidyuesday du 4 juin 2024 (<https://github.com/rfordatascience/tidyuesday/blob/main/data/2024/2024-06-04/readme.md>).

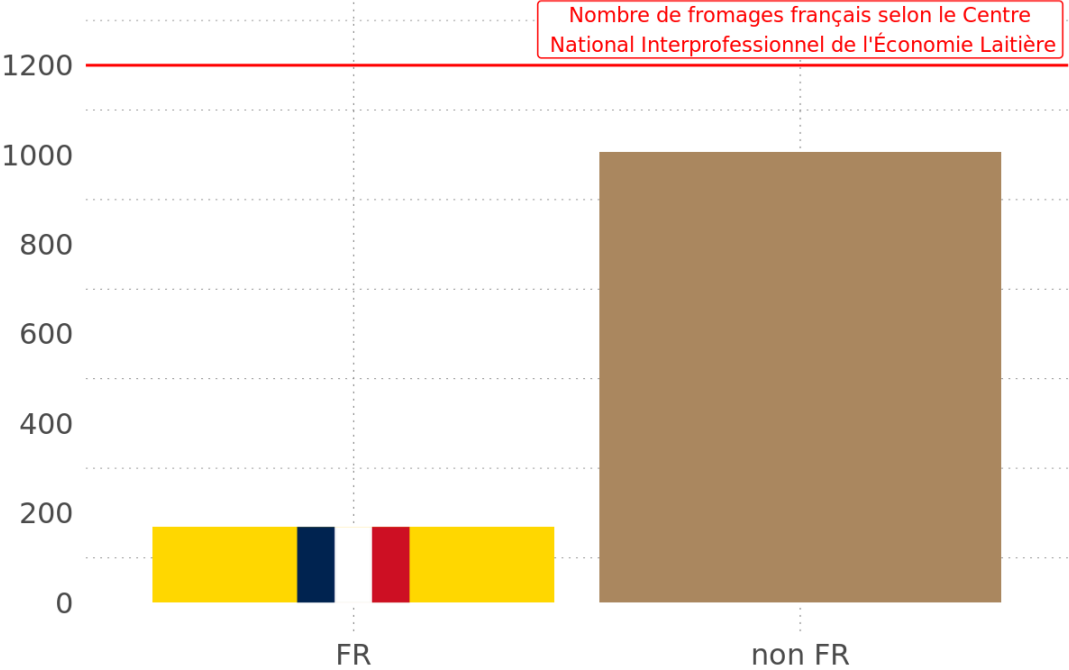
2. Premières observations

2. PREMIERES OBSERVATIONS

Notre jeu de données contient des informations sur **1187** fromages.

On constate tout de suite une grande infamie, **un biais terrible, un crime de lèse fromajesté** :

Nombre de fromages référencés sur cheese.com par origine

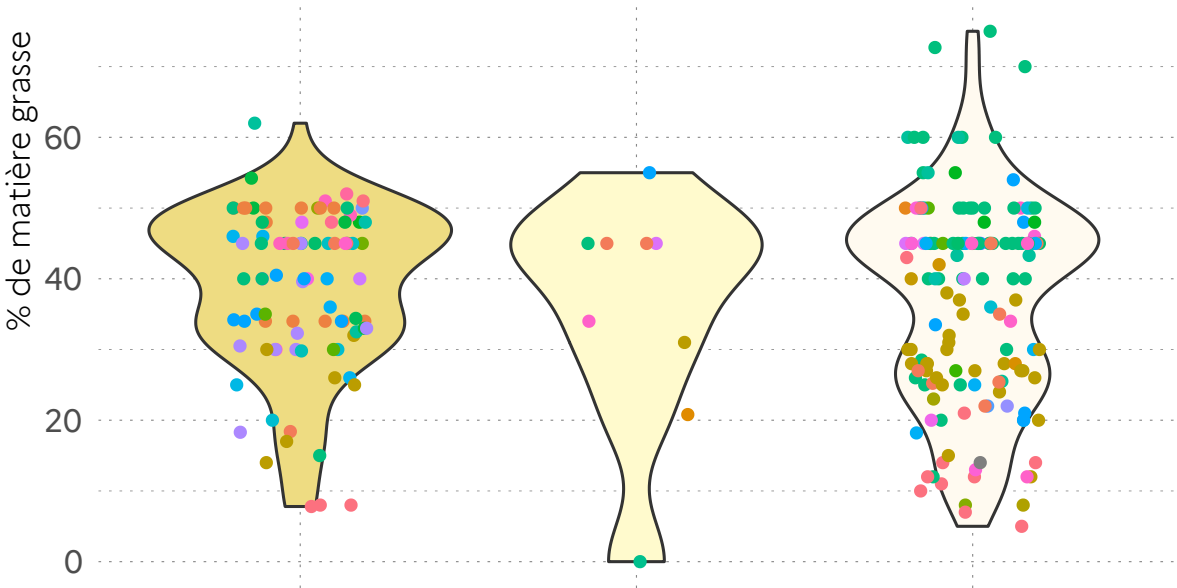


3. Fromage 2.0

a - Widget html

% de matière grasse par type de pâte de fromage

Monde entier



dure

ferme

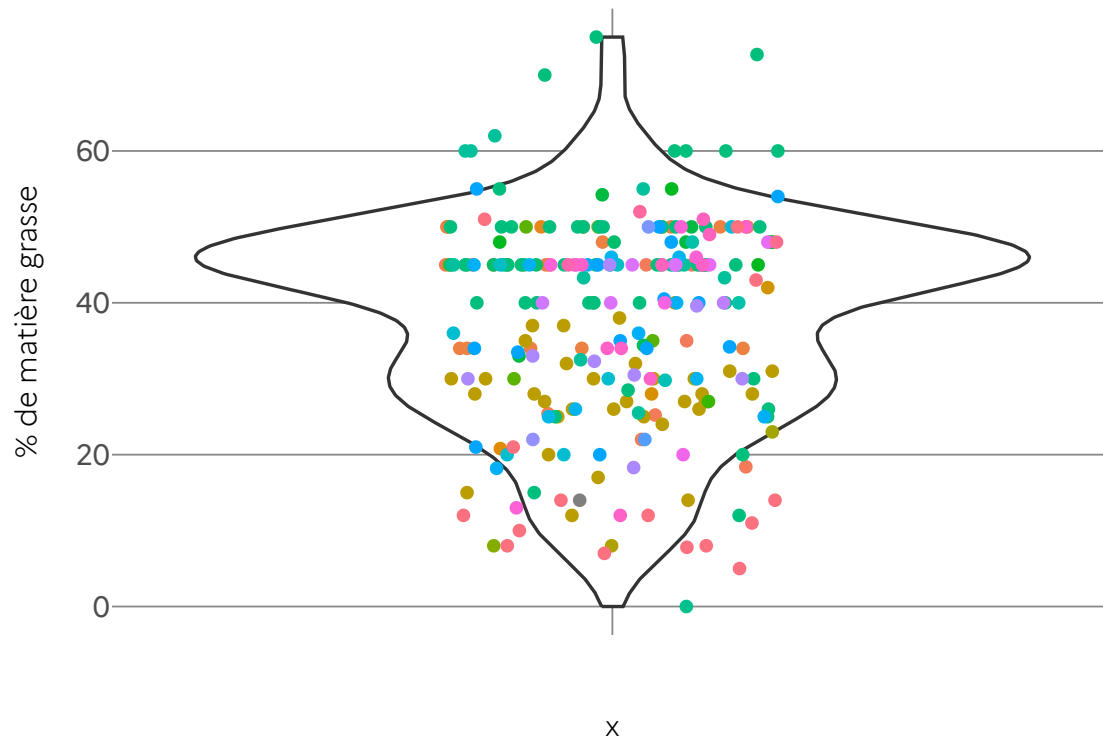
molle

Type de pâte

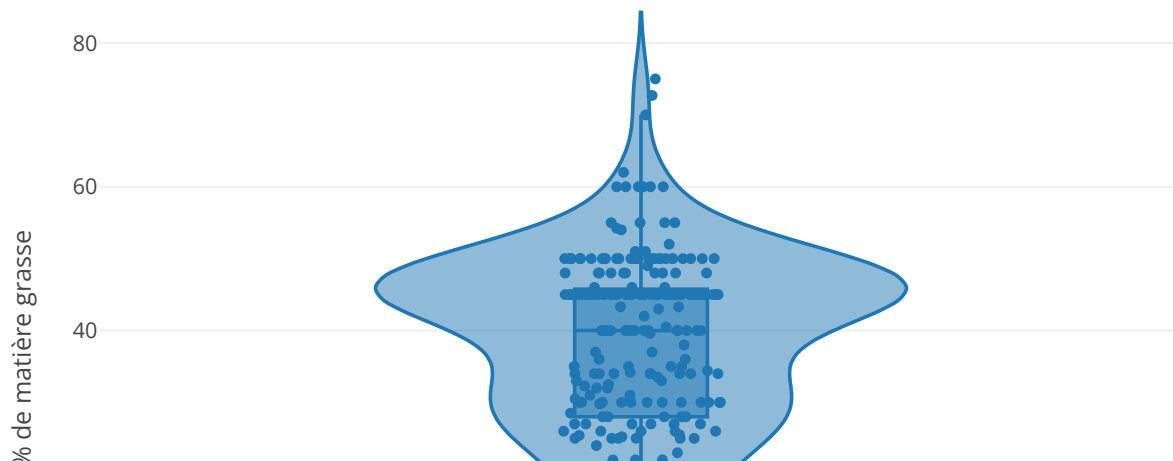
b - Crosstalk, de la datafromage à partager

Choisissez un pays

% de matière grasse dans les fromage



% de matière grasse dans les fromages



8.5 Interactivité complète, l'application Shiny

embarquée

À condition de disposer d'un serveur R pour déployer son document, il est possible d'intégrer des objets interactifs Shiny (inputs et outputs) voire une application Shiny complète dans un Rmarkdown.

Dans votre yaml, il faut inclure la ligne `runtime: shiny`, pour reprendre l'exemple du chapitre 6, cela donne :

```
---
title: "mon_premier_document"
author: "Moi"
date: "31/10/2022"
runtime: shiny
output: html_document
params:
  annee: 2022
  region: Bretagne
  date: !r lubridate::today()
---
```

Vous pouvez voir que Rstudio vous propose désormais “Run document” au lieu de “Knit”. Il comprend qu’il doit désormais utiliser un moteur shiny pour permettre la visualisation du document. Attention, ce moteur “shiny” n’est pas embarqué dans le HTML produit, cela signifie qu’une installation de R doit être accessible du lecteur du document produit.

À partir de là, vous pouvez inclure directement des inputs et outputs Shiny dans votre document mais il y a un twist : contrairement à une application Shiny avec une logique UI / serveur où vous devez prévoir des placeholders dans l’UI pour vos sorties de serveur, dans la version Rmarkdown vous pouvez directement appeler les fonctions de la famille `renderXXX()`.

Application Shiny classique :

```
library(shiny)

ui <- bootstrapPage(

  selectInput("input_utilisateur", ...),

  plotOutput("mon_graph", ...)

)

server <- function(input, output) {

  output$mon_graph <- renderPlot({
    ggplot(...)
  })

}

shinyApp(ui = ui, server = server)
```

La même chose en Rmarkdown :

```
---  
runtime: shiny  
output: html_document  
---  
  
```${r eval = FALSE}  

selectInput("input_utilisateur")

```${r eval = FALSE}  
renderPlot({  
  
  ggplot()  
  
})  
```${r eval = FALSE}
```

La syntaxe s'en trouve simplifiée, c'est la structure du document Rmarkdown qui permet de gérer le placement des différents éléments dans l'interface.

Une autre option consiste à directement intégrer le code de l'application Shiny tel quel dans le document Rmarkdown :

```

runtime: shiny
output: html_document

```{r eval = FALSE}

library(shiny)

ui <- bootstrapPage(

  selectInput("input_utilisateur", ...),

  plotOutput("mon_graph", ...)

)

server <- function(input, output) {

  output$mon_graph <- renderPlot({
    ggplot(...)
  })

}

shinyApp(ui = ui, server = server, options = list(height = 500))

```

```

L'argument "height" en option permet d'indiquer la place (ici par défaut en pixel) que doit occuper l'application dans le document.

La réciproque est vraie, il est possible d'inclure des documents markdown dans une application Shiny (souvent utilisé pour la page d'accueil) en utilisant la fonction `includeMarkdown("mon_markdown.md")` à l'intérieur de l'UI.

Il est donc possible d'inclure dans une app Shiny un markdown qui lui même contient une application Shiny... 🤖

# Chapitre 9 Mettre en ligne les HTML produits avec Rmarkdown

Les fichiers HTML ne peuvent pas simplement venir compléter nos sites institutionnels gérés avec Spip/Giseh. Pour les partager en interne, les déposer sur un serveur bureautique suffit. En revanche, pour les partager avec l'extérieur, il est souvent nécessaire d'héberger les documents sur un serveur dédié.

## 9.1 Publier sur un serveur HTML statique

Le service informatique du pôle ministériel (DNUM/Sous-direction des méthodes et des services de plateforme) propose une offre d'hébergement gratuite pour toute entité du pôle ministériel Ecologie qui la demande.

Il faut au préalable avoir la validation d'une nouvelle url pour votre organisme par la direction de la communication.

Cette offre est décrite sur le catalogue des offres spote : <https://spote.developpement-durable.gouv.fr/offre/webstatic-publication-de-sites-de-contenus-statiques>.

La DNUM procède en deux temps : 1. elle vous livre le serveur et ses identifiants d'utilisation et 2. elle vous indique que le certificat pour bénéficier de la norme https est actif.

Une fois le serveur configuré par la DNUM, on envoie les fichiers HTML à publier par FTP, par exemple avec le logiciel Filezilla.

Exemple de serveur statique : <https://dreal.statistiques.developpement-durable.gouv.fr/>

Mode opératoire pour déposer sur ce serveur : [https://gitlab.com/rdes\\_dreal/propres\\_rpls/-/wikis/Diffusion-des-publications](https://gitlab.com/rdes_dreal/propres_rpls/-/wikis/Diffusion-des-publications)

## 9.2 Publier grâce aux fonctionnalités de déploiement

# continu d'une forge git

## 9.2.1 Exemple : le parcours R

C'est l'option retenue pour publier les supports de formations du parcours R.

Chaque formation est déposée sur un répertoire github : cela permet d'avoir le suivi des versions, le travail collaboratif sur le contenu source, et surtout le déploiement public et automatique de chaque modification vers [https://mtes-mct.github.io/parcours\\_r\\_module\\_publication\\_rmarkdown/](https://mtes-mct.github.io/parcours_r_module_publication_rmarkdown/) par exemple pour le présent module 6.

## 9.2.2 Concepts

Github et Gitlab sont deux plateformes web concurrentes, dites forge de développement, utilisées pour le travail collaboratif sur le code. Pour les utiliser, il faut versionner ses projets R avec git. Git un exécutable qui s'installe sur son PC, permettant le suivi de version des scripts et permettant le dialogue avec ces forges web.

## 9.2.3 Quelle forge choisir ?

Le parcours-R est hébergé sur github car à ses débuts, c'était l'offre la plus populaire.

Le ministère dispose désormais d'une instance officielle de gitlab : <https://gitlab-forge.din.developpement-durable.gouv.fr/>. Chaque service du pôle ministériel peut disposer d'un dépôt gitlab-forge après avoir formulé une demande officielle, cf fiche spote <https://spote.developpement-durable.gouv.fr/offre/gitlab-forge-logicielle-et-livraison-continue>.

Chaque agent disposant d'un compte CERBERE peut rejoindre un groupe gitlab-forge ou y disposer d'un dépôt personnel.

Si besoin de collaborer avec des personnes qui n'ont pas de compte CERBERE, il est possible de faire une demande pour 'cerberiser' son adresse mail et l'autoriser à écrire sur un dépôt gitlab-forge.

En attendant, gitlab.com offre une version freemium accessible de tous. À la fin de la collaboration, on peut facilement réimporter le travail de gitlab vers gitlab-forge.

## 9.2.4 Comment on fait ?

Pour configurer git et RStudio, suivre les fiches tutorielles : <https://dreal-pdl.gitlab-pages.din.developpement-durable.gouv.fr/csd/geodata4apps.fiches/decouverte.html>

Une fois la configuration de git achevée, imaginons que vous vouliez publier un fichier publi.html généré à partir d'un fichier publi.Rmd. Voici un pas à pas de base :

1. dans votre projet RStudio, placez les fichiers html (et leurs éventuelles dépendances) que vous voulez publier dans un dossier nommé 'public'. S'il n'y a qu'un fichier, nommez les index.html, c'est intéressant pour raccourcir l'url.
2. toujours dans votre projet RStudio, lancer `usethis::use_git()` dans la console pour activer le tracking de git pour ce projet. RStudio vous propose d'enregistrer une version, sorte de premier point de restauration, de votre projet :

```
> usethis::use_git()

✓ Setting active project to "C:/Users/juliette.engelaere/Docum
✓ Initialising Git repo.
✓ Adding ".Rproj.user", ".Rhistory", ".RData", ".httr-oauth",
i There are 10 uncommitted files:
• ..gitignore
• cagnotte.R
• data/
• diaporama.html
• diaporama.Rmd
• diaporama_files/
• leaflet.R
• libs/
• xaringan-themer.css
• yoyo.Rproj

! Is it ok to commit them?

1: Absolutely
2: Not now
3: No

Sélection : |
```

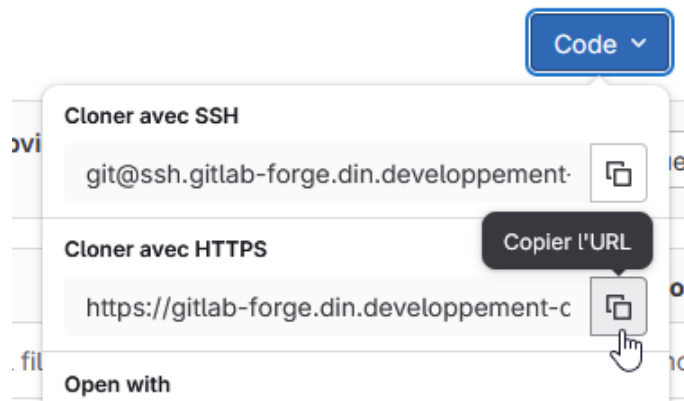
Il faut accepter (Ici : appuyer sur 1 + entrée).

3. Acceptez de redémarrer R pour que les fonctionnalités git s'activent dans votre projet Rstudio (nouveau panneau)

A cette étape, votre projet a une version, prête à être partagée sur une forge git, reste à indiquer où.

1. Depuis votre dépôt gitlab-forge, créer un nouveau projet **vide** (décocher "Initialiser le dépôt avec un README").

2. Copier l'url https de ce projet



3. Retourner dans RStudio pour pointer votre projet R vers ce projet git, pour cela, lancer dans la console :

```
usethis::use_git_remote(url = "https://gitlab-forge.din.developpement-durable.gouv.fr/groupe/projet.git")
```

4. Dernière étape pour partager le code : il faut connecter les branches et réaliser le premier 'push', en lançant dans la console R :

```
system("git push -u origin main")
```

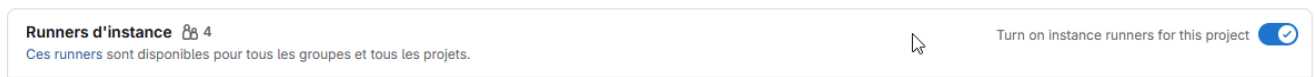
votre projet est alors envoyé sur la forge.

5. Intégration continue

Par convention ce qui se situe dans le dossier public, peut être publié automatiquement par gitlab sur gitlab-pages si l'intégration continue est active.

Pour l'activer, 2 conditions :

- un bouton à cocher dans les paramètres du dépôt (à retrouver au niveau du menu Parametres / CI-CD / Runners)



- et un fichier .gitlab-ci.yml à mettre à la racine du projet.

Voici un exemple minimaliste de script à placer dans votre projet sous le nom `.gitlab-ci.yml` qui utilise le répertoire public et le rend accessible sur 'Pages'.



```
pages:
 stage: deploy
 script:
 - echo 'Deploiement en cours'
 artifacts:
 paths:
 - public
 only:
 - master
```

## 9.3 Exercice 8 (facultatif)

Expérimenter gitlab pages

# Chapitre 10 Pour aller plus loin

Derrière Rmarkdown aujourd'hui existe tout un écosystème permettant de mixer des traitements réalisés en R et du texte dans des formats très différents, que ce soit en matière de technologie (html, pdf, ...) ou de type de rendu (diaporama, rapport, livre, note, cv, ...).

## 10.1 Les documents Quarto

Un document Quarto est un format de document qui permet de combiner du texte, du code et des visualisations dans un seul fichier, comme un Rmarkdown en somme. En revanche, Quarto n'est ni dépendant ni exclusif à R et Rstudio. Il ne s'agit pas d'un package de R, il faut l'installer "à part" (mais il est inclus dans l'installation de Rstudio).

Les différences principales sont :

- l'amélioration du support multi-langage (R, Python, Javascript, Julia) et multi-moteur (knitr, Jupyter, Observable) ;
- un rendu amélioré : Quarto offre des options de rendu plus riches et plus flexibles, y compris des thèmes et des styles personnalisés pour les documents ;
- l'interactivité : Quarto facilite l'intégration d'éléments interactifs, comme des widgets et des visualisations interactives, directement dans les documents (à condition de les coder dans leur langage d'origine) ;
- la publication : Quarto propose des options intégrées pour publier des documents en tant que sites web, livres électroniques ou présentations, avec une configuration minimale ;
- les fonctionnalités futures : les concepteurs du format ont annoncé qu'ils allaient continuer à assurer le support du format Rmd, mais que de nouvelles fonctionnalités ne seraient apportées que sur le Quarto.

Si vous voulez tester la transition, dans la majorité des cas, un copier / coller direct de votre Rmd dans un qmd fonctionnera ! Et pour un test rapide, créez un nouveau document dans Rstudio en choisissant le format "Quarto document" proposé et laissez-vous guider.

Vous trouverez toute la documentation, des exemples mais surtout des guides très complets pour s'autoformer directement [sur le site de Quarto](#) et également sur le [site de formation à R du ministère de l'Agriculture](#).

## 10.2 Formats de sortie

Il existe de nombreux packages permettant d'obtenir des formats de sortie spécifiques.

Utiliser des formats de sorties de type bureautiques est très utile si :

- on souhaite tirer partie de modèles bureautiques, conformes à une charte graphique que l'on souhaite respecter,
- on souhaite faire éditer le document produit par des non-Ristes.

Cela peut s'avérer nécessaire pour faire relire et valider le document avant sa publication, mais attention ! Il faudra dans un second temps reporter dans Rmarkdown les modifications opérées en dehors, pour tirer pleinement partie de la reproductibilité offerte par Rmarkdown.

### 10.2.1 odt\_document

Parmi les types de sortie possibles, `{rmarkdown}` propose le format Libre Office Writer .odt Il repose pour cela sur Pandoc qui traite la conversion de markdown (fichier .md) en document Libre Office Writer (fichier .odt). Depuis R, cette conversion offre un nombre limité de paramètres de personnalisation.

Paramètres de la fonction `odt_document()` :

Ce qui donne pour l'entête YML

```
odt_document(
 number_sections = FALSE,
 fig_width = 5,
 fig_height = 4,
 fig_caption = TRUE,
 template = "default",
 reference_odt = "default",
 includes = NULL,
 keep_md = FALSE,
 md_extensions = NULL,
 pandoc_args = NULL
)
```

```

title: "mon_premier_document"
author: "Moi"
date: "31/10/2025"
output:
 odt_document:
 number_sections: false
 fig_width: 5
 fig_height: 4
 fig_caption: true
 reference_odt: "chemin/vers/le/modele.odt"

```

- `number_sections` : TRUE/FALSE, sert à numéroter (ou pas) les titres.

- `fig_width` / `fig_height` sert à indiquer la largeur / hauteur par défaut (en pouces) pour les figures.
- `fig_caption` : TRUE/FALSE pour afficher des figures avec des légendes à partir des paramètres de `chunks` ( `fig.cap = "légende de la figure"` ).
- `template` : modèle Pandoc à utiliser pour le rendu, 3 possibilités : 1. laisser à "default" pour utiliser le modèle par défaut du package `rmarkdown`, 2. indiquer NULL pour revenir au modèle intégré de Pandoc ou 3. indiquer un chemin d'accès vers un modèle pandoc personnalisé. Consulter la [documentation de Pandoc](#) pour plus de détails sur la création de modèles personnalisés.
- `reference_odt` : pour spécifier un fichier odt dont on souhaite réutiliser les styles. Pour un résultat optimal, le fichier odt de référence doit être une version modifiée d'un fichier odt généré avec pandoc.
- `includes` : liste nommée de contenu supplémentaire à inclure dans le document comme un en-tête ou un bas de page (à créer à l'aide de la [fonction](#) `includes()` ).
- `pandoc_args` : options de ligne de commande supplémentaires à transmettre à pandoc.

Une manière rapide de **styler** le fichier compilé est de le générer une fois, de l'éditer avec libre office pour y [adapter les styles de titres, listes...](#) au rendu souhaité.

On peut également coder les styles directement en xml cf. [ce tuto en français](#)

La gestion des **tableaux** est parfois moins souple qu'en HTML/PDF, mais on peut obtenir quelque chose de propre. Les tableaux simples sont traités correctement avec `knitr::kable()` en spécifiant le format "pandoc".

```
knitr::kable(head(iris), format = "pipe")
```

Il ne faut pas utiliser les fonctions spécifiques au format HTML (comme `kable_styling()` , `column_spec()` ... ) car elles ne sont pas supportées. Mais `{kableExtra}` sait aussi générer des tableaux LaTeX et les fonctions compatibles avec le format PDF le seront généralement aussi pour ODT.

Les **graphiques** ne sont pas automatiquement redimensionnés comme dans un format de sortie HTML, il faut rester dans les limites de largeur du corps de la page (environ 6 pouces pour le modèle par défaut).

## 10.2.2 Officer et Officedown



Les packages officer et officedown permettent de générer des documents texte ou des présentations en format MSOffice. <https://ardata-fr.github.io/officeverse/index.html>

Exemple [https://gitlab-forge.din.developpement-durable.gouv.fr/dreal-pdl/csd/crte/-/blob/master/inst/rmarkdown/templates/portrait\\_territoire/skeleton/skeleton.Rmd?ref\\_type=heads&plain=1](https://gitlab-forge.din.developpement-durable.gouv.fr/dreal-pdl/csd/crte/-/blob/master/inst/rmarkdown/templates/portrait_territoire/skeleton/skeleton.Rmd?ref_type=heads&plain=1)

## 10.2.3 Présentations diaporama HTML

Xaringan Pour des présentations de ninja

<https://support.posit.co/hc/en-us/articles/360004672913-Rendering-PowerPoint-Presentations-with-the-RStudio-IDE>

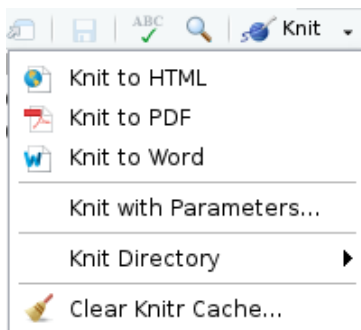
## 10.3 Mettre des résultats en cache pour gagner du temps de compilation

Compiler un document R Markdown peut être long, car il faut à chaque fois exécuter l'ensemble des blocs de code R qui le constituent.

Pour accélérer cette opération, knitr propose un système de mise en cache : les résultats de chaque bloc sont enregistrés dans un fichier et à la prochaine compilation, si le code et les options du bloc n'ont pas été modifiés, c'est le contenu du fichier de cache qui est utilisé, ce qui évite de ré-exécuter ce code R.

On active ou désactive la mise en cache des résultats pour chaque chunk avec l'option `cache = TRUE` ou `cache = FALSE`, et on peut aussi désactiver totalement la mise en cache pour le document en ajoutant `knitr::opts_chunk$set(cache = FALSE)` dans le premier chunk de 'setup'.

Il faut manier cette option avec beaucoup de prudence. Le système de cache peut fausser les résultats s'il n'est pas utilisé avec maîtrise, par exemple s'il n'est pas vidé alors que les données source changent. Dans ce cas les résultats de certains blocs ne seront pas mis à jour s'ils sont présents en cache. On peut vider le cache du document, ce qui forcera un re-calcul de tous les blocs de code à la prochaine compilation. Pour cela, ouvrir le menu Knit (la pelote bleue) et choisir Clear Knitr Cache... (le balai) :



(Source : [guide analyse-R de Joseph Larmarange](#))

## 10.4 Rendre le paramétrage interactif

dernière slide de l'atelier 2

## 10.5 Transformer les sous-titres en onglets

Ajouter l'attribut `.tabset` au titre transforme tous les sous-titres en onglets au lieu de sous-sections :

**Code :**

**Résultat :**

```
Exemple d'onglets {.tabset}
```

```
Onglet 1
```

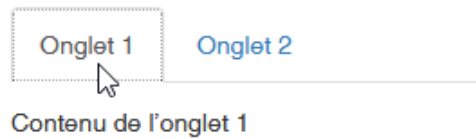
Contenu de l'onglet 1

```
Onglet 2
```

Contenu de l'onglet 2

```
{.unlisted .unnumbered}
```

## Exemple d'onglets



Pour finir un tabset, commencer un nouveau titre du niveau supérieur vide `### {.unlisted .unnumbered}` pour continuer avec des paragraphes réguliers.

Plus d'options d'onglets sont présentées dans le livre de recettes Rmarkdown :  
<https://pkg.yihui.org/rmarkdown-cookbook/html-tabs>.

Ce paramétrage en onglets ne fonctionne pas dans les projets de type bookdown car les titres sont déjà mobilisés pour construire le menu de gauche.

## 10.6 Bibliographie

Voici quelques éléments bibliographiques pour en savoir plus sur R Markdown mais aussi tous les packages et logiciels en lien avec ce sujet :

- [Le guide définitif de Rmarkdown](#)
- [Le guide de référence](#)
- [La syntaxe Markdown sur laquelle s'appuie rmarkdown](#)
- [Pour en savoir plus sur PANDOC](#)
- [Pour en savoir plus sur TinyTeX](#)
- [Une vidéo en anglais pour aller plus loin dans l'utilisation de R Markdown](#)
- et évidemment les documentations de tous les packages cités !

## Chapitre 11 Pour aller plus loin sur Pagedown

Pagedown est une implémentation pour Rmarkdown de [paged.js](https://github.com/jgm/paged.js), qui permet de réaliser des documents html paginés.



C'est un outil puissant pour concilier les fonctionnalités HTML (donc mise en formes CSS) et la possibilité d'impression PDF, galerie :



## Au 1<sup>er</sup> janvier 2026, + 6,8 % de personnes détenues sur un an

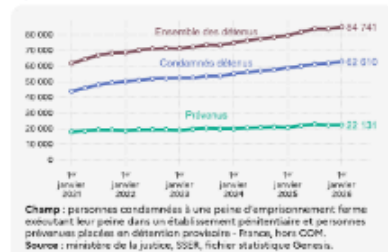
Evan Le Bihan  
Jérôme Moreau

Les statistiques de milieu fermé, publiées tous les trimestres, décrivent la population pénitentiaire selon différents axes : statut pénal, caractéristiques sociodémographiques, types d'infraction, durée de la peine, etc. Cette publication présente quelques indicateurs issus de ces statistiques sur un périmètre France hors collectivités d'outre-mer (COM), sauf mention contraire. L'ensemble des indicateurs est disponible sur la [page dédiée](#) à ces statistiques, y compris ceux relatifs aux COM.

### 86 100 personnes détenues en France, y compris COM

Au 1<sup>er</sup> janvier 2026, 86 100 personnes sont détenues en France, y compris COM. Parmi elles, 1 400 personnes sont détenues au sein d'un établissement des COM (Nouvelle-Calédonie et Polynésie française). Ainsi, 84 700 personnes sont détenues en France, hors COM. Parmi ces dernières, 62 600 sont condamnées et 22 100 sont prévenues. Le nombre total de détenus augmente de 6,8 % sur un an et de 1,5 % sur un trimestre. Sur 12 mois, le nombre de condamnés est en hausse de 6,6 %, alors que celui des prévenus s'accroît de 7,5 %. Au cours du quatrième trimestre, 19 900 personnes ont été placées en détention.

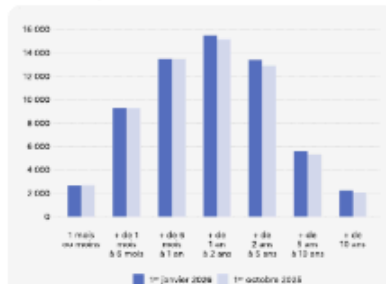
Figure 1. Évolution du nombre de détenus



### 2 600 personnes condamnées détenues libérables dans le mois

Au 1<sup>er</sup> janvier 2026, 54 300 personnes condamnées sont détenues pour une durée de 5 ans ou moins, soit + 5,8 % sur une année et + 1,6 % par rapport à la fin du trimestre précédent. Parmi celles-ci, 2 600 vont purger leur peine dans le mois. À l'opposé, 2 200 personnes doivent exécuter leur peine pendant au moins 10 ans. Les durées de peine restantes ne tiennent pas compte d'éventuels diminutions ou allongements ultérieurs (réduction de peine, nouvelle condamnation, etc.).

Figure 2. Condamnés détenus par durée de peine ferme restante (reliquat)

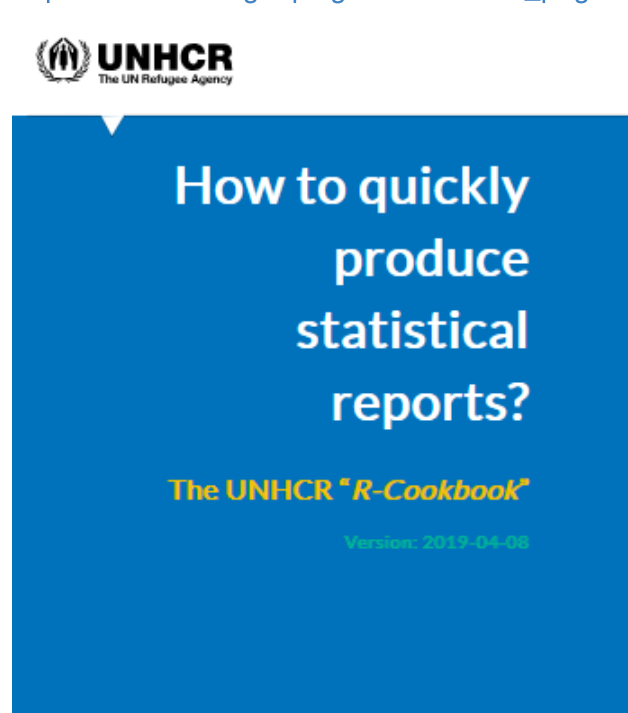


**Remarque :** les durées de peine ferme restantes ne sont pas renseignées pour 0,1 % des personnes au 1<sup>er</sup> janvier 2026 et 0,3 % des personnes au 1<sup>er</sup> octobre 2025.

**Champ :** personnes condamnées à une peine d'emprisonnement ferme exécutant leur peine dans un établissement pénitentiaire - France, hors COM.

**Source :** ministère de la justice, SSER, fichier statistique Genedis.

- [https://edouard-legoupil.github.io/unhcr\\_paged/#graphics-with-style](https://edouard-legoupil.github.io/unhcr_paged/#graphics-with-style)



- [https://dreal.statistiques.developpement-durable.gouv.fr/parc\\_social/2020/www/export\\_demarchePropre.pdf](https://dreal.statistiques.developpement-durable.gouv.fr/parc_social/2020/www/export_demarchePropre.pdf)

```
knitr::include_url("https://dreal.statistiques.developpement-durable.gouv.fr/parc_social/2020/www/
```



## 11.1 Paged.js



[paged.js](https://www.w3.org/TR/css-page-3/) est une bibliothèque javascript visant à mettre en oeuvre [les propriétés css dédiées aux médias paginés](<https://www.w3.org/TR/css-page-3/>) du W3C.

Ces spécifications visent à pouvoir réaliser des documents prêt pour l'impression avec les technologies du web (html, css, js).

Ces spécifications sont toujours en draft pour le moment au sein du W3C, donc pas vraiment reconnues par les principaux navigateurs. D'où le besoin de cette bibliothèque javascript pour pouvoir les mettre en oeuvre.

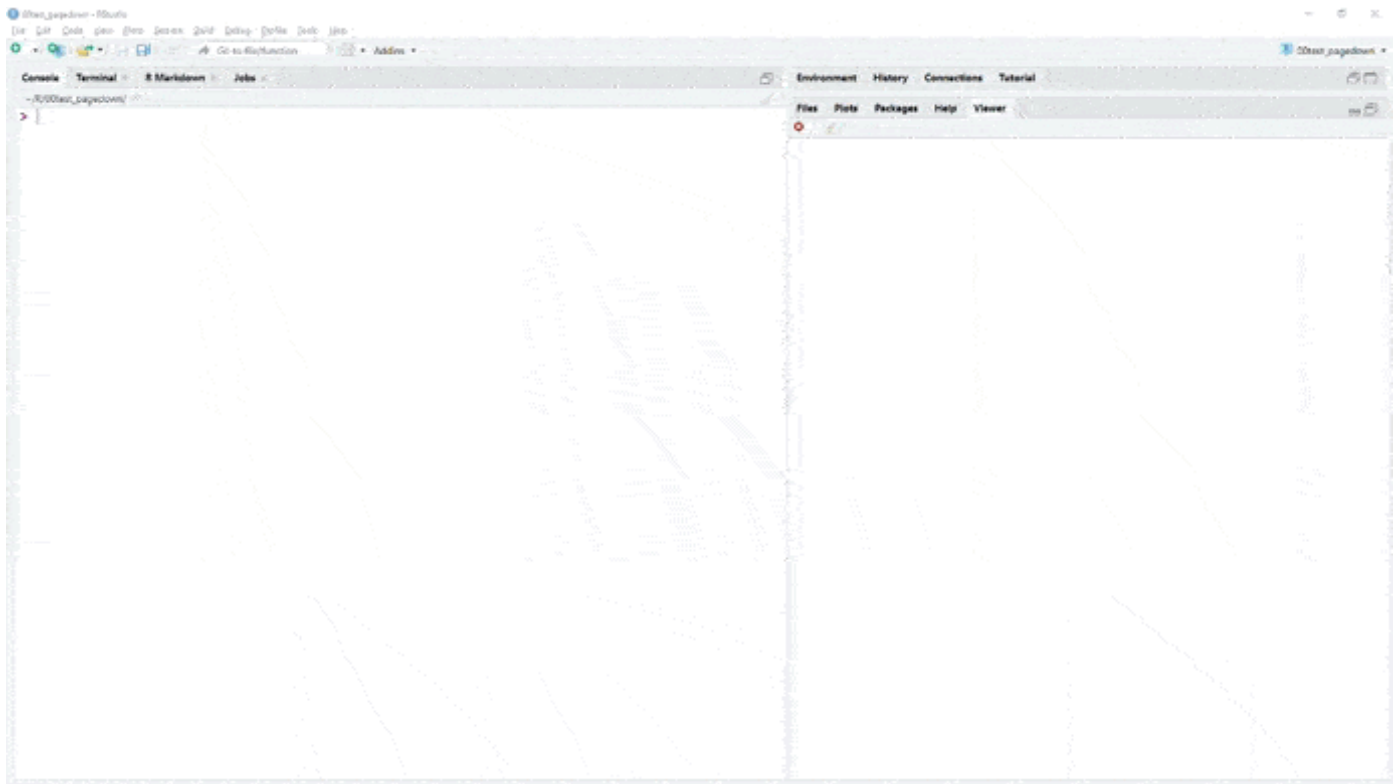
## 11.2 Démarrer avec Pagedown

Pour installer `{pagedown}` depuis le CRAN :

```
install.packages('rstudio/pagedown')
```

Depuis Rstudio, cela vous apporte un nouveau type de document rmarkdown, accessible depuis `File -> New File -> Rmarkdown... -> From template -> Paged HTML document` .

Une fois celui ci créé, vous pouvez cliquer sur knit de l'interface de Rstudio. Cela vous compilera le document par défaut en html, qui sera accessible dans le répertoire du projet et visible par défaut dans le viewer.



La structure du yaml est relativement proche d'un document Rmarkdown classique.

A noter toutefois une option utile à retenir, la balise `knit: pagedown::chrome_print` qui vous permet de compiler directement votre document en pdf grâce à la fonction `pagedown::chrome_print()` livrée dans le package.

Cette fonction peut être utilisée par ailleurs pour imprimer tout document html en pdf ou en format image. Elle utilise la technologie d'impression de google chrome.

```
title: "A Multi-page HTML Document"
author: "Yihui Xie and Romain Lesur"
date: "2026-09-02"
output:
 pagedown::html_paged:
 toc: true
 \# change to true for a self-contained document, but it'll be a litte slower for Pandoc to ren
 self_contained: false
\# uncomment this line to produce HTML and PDF in RStudio:
\#knit: pagedown::chrome_print
```

## 11.3 Configurer votre yaml

Les documents html paginés étant destinés à être imprimés, l'entête yaml d'un document Pagedown contient des options du yaml spécifiques. En voici quelques unes intéressantes à connaître :

- Pour modifier le titre du sommaire du document :

```
toc-title: sommaire
```

- Pour ajouter et configurer une liste de tableaux et de graphiques dans le sommaire.

```
lot: true
lot-title: "Tableaux"
lof: true
lof-title: "Graphiques"
```

Si vous ne voulez pas que les graphiques et tableaux apparaissent dans le sommaire, vous pouvez utiliser `lot-unlisted: true` .

- Pour modifier le préfixe du titre du chapitre.

```
chapter_name: "Chapitre\\ "
```

Vous pouvez ensuite insérer un chapitre en utilisant `# Ceci est un titre de chapitre {.chapter}`

- Pour transformer automatiquement un lien url dans votre document en note de bas de page.

```
links-to-footnotes: true
```

Ainsi `[CRAN](https://cran.r-project.org/)` sera traduit comme `CRAN^[https://cran.r-project.org/)]` .

- `front_cover` et `back_cover` pour ajouter une image pour la première de couverture et la 4e de couverture :

```
pagedown::html_paged:
 front_cover: !expr system.file("help","figures","lter_penguins.png", package = "palmerpenguins")
 back_cover: https://www.r-project.org/Rlogo.png
```

On utilise le préfixe `!expr` pour insérer du code R créant en sortie une image.

## 11.4 Quelques balises importantes à connaître

`{.page-break-before}` et `{.page-break-after}` sont des classes CSS qui insèrent un saut de page avant/après le paragraphe auquel la balise est attachée.

```
Nouveau chapitre après un saut de page {.page-break-before}
```

## 11.5 Modifier le CSS

Apprendre à customiser un document pagedown va vous demander d'investir sur l'apprentissage du CSS en général et de pagedjs en particulier pour construire un template ad hoc.

C'est un investissement en soit, qui pourra vous être utile de la même façon que la gestion de la mise en page d'un document bureautique.

Vous pouvez aussi centraliser cet investissement pour votre équipe sur une ou deux personnes, ou travailler avec votre service web sur vos projets.

Quelques ressources pour apprendre le CSS :

- [Le site de la fondation Mozilla](#)
- [Le site du W3C](#)

Voici déjà quelques recettes de base pour commencer à modifier votre document.

Première étape : importer les css par défaut de pagedown dans votre document.

```
files <- c("default-fonts", "default-page", "default")
from <- pagedown::pkg_resource(paste0("css/", files, ".css"))
to <- c("custom-fonts.css", "custom-page.css", "custom.css")
file.copy(from = from, to = to)
```

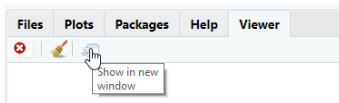
Ajouter une référence à ces fichiers dans le yaml :

```

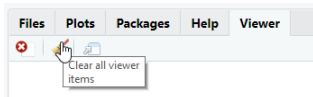
output:
 pagedown: :html_paged:
 css:
 - custom-fonts.css
 - custom-page.css
 - custom.css
 toc: true
 self_contained: false

```

- Lancer dans la console `xaringan::inf_mr()` , qui permet de compiler un document Rmarkdown en ayant un aperçu live du rendu.
- Appuyer ensuite sur `Show in new window` pour consulter le document sur votre navigateur par défaut.



- Puis sur `Clear all viewer item` , pour que vos modifications puissent s'afficher en live.



## 11.6 Configurer votre CSS

Vous pouvez commencer à modifier vos fichiers css et voir le résultats immédiatement.

Les 3 fichiers correspondent aux éléments suivants :

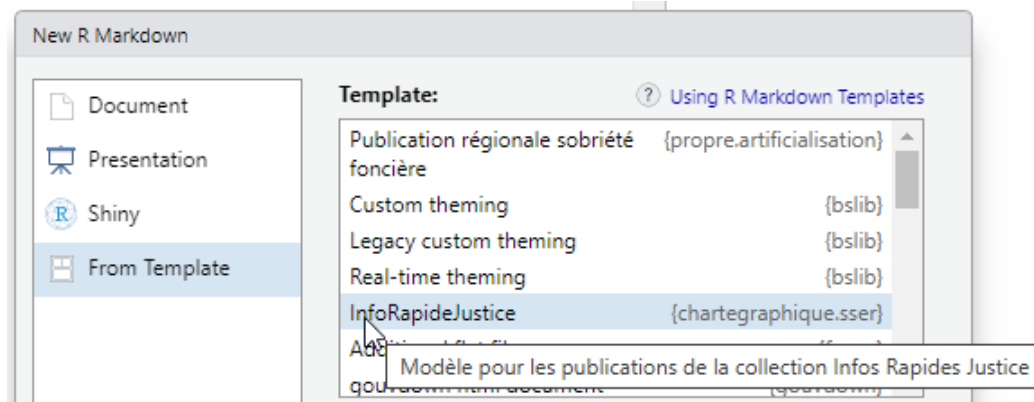
- `custom-fonts.css` correspond aux polices de caractères
- `custom-page.css` correspond aux paramètres de la page (format de la page, orientation,...)
- `custom.css` intègre entre autre la façon dont les pages sont affichées à l'écran (couleur de fond, espacement entre les pages...)

## 11.7 Exemple d'usage : le modèles de 4 pages Info rapides Justice du SSER

Le Rmd modèle s'installe avec le package `{chartegraphique.sser}` :

```
devtools::install_gitlab(repo="sser/charte graphique.sser@master",
 host="https://git.lab.sspcloud.fr",
 dependencies = TRUE,
 upgrade = FALSE)
```

Après avoir redémarré R, on a une nouvelle entrée dans le menu `File / New File / R Markdown... / From Template`



Cliquer sur cette nouvelle entrée permet d'accéder au document, puis de le compiler pour accéder au résultat.